

**A MULTI-CRITERIA STUDENT-TUTOR MATCHMAKING SYSTEM USING
WEIGHTED SCORING AND AVAILABILITY OPTIMIZATION**

A Final Year Project Presented by

ANUMUDU UDOCHUKWU SAMUEL

20211259202

AND

OBI MICHAEL CHIMAObI

20211265722

Submitted to

THE DEPARTMENT OF SOFTWARE ENGINEERING

SCHOOL OF INFORMATION AND COMMUNICATION TECHNOLOGY

FEDERAL UNIVERSITY OF TECHNOLOGY OWERRI

Supervised By

Engr. Dr. A. I. Erike

IN PARTIAL FULFILLMENT OF THE REQUIREMENTS FOR THE AWARD OF

BACHELOR OF TECHNOLOGY (B. TECH) IN SOFTWARE ENGINEERING

JULY 2026

CERTIFICATION

The undersigned certify that this project, A Multi-Criteria Student-Tutor Matchmaking System Using Weighted Scoring and Availability Optimization, was carried out by Anumudu Udochukwu Samuel (20211259202) and Obi Michael Chimaobi (20211265722) of the Department of Software Engineering, School of Information and Communication Technology, Federal University of Technology Owerri, in partial fulfilment of the requirements for the award of Bachelor of Technology (B.Tech) in Software Engineering.

_____	_____
Engr. Dr. A. I. Erike	Date
Project Supervisor	

_____	_____
Head, Department of Software Engineering	Date

_____	_____
Dean, School of Information and Communication Technology	Date

DEDICATION

This work is dedicated to God Almighty, for the grace and strength that sustained it, and to our parents and families, whose support and sacrifice made it possible.

ACKNOWLEDGEMENT

A project of this kind is never the work of two people alone. We are grateful to our supervisor, Engr. Dr. A. I. Erike, for the direction and the patience that shaped this work from proposal to submission, and to the lecturers of the Department of Software Engineering, School of Information and Communication Technology, Federal University of Technology Owerri, for the grounding that made it possible. Our thanks go also to the students and tutors whose experience of tutoring informed the design, and to our families and friends for the support that carried the work through its longer stretches. Whatever is good in this project is owed to them; the errors that remain are ours.

ABSTRACT

Student-to-tutor matching is a multi-criteria decision problem, yet most tutoring platforms match on subject and availability alone. This project designed, implemented and tested a multi-criteria matchmaking system for secondary-school tutoring. Compatibility comes from a composite score with four bounded sub-score functions (academic, preference, scheduling and fairness) weighted by the student, and assignment runs as a priority-queue greedy algorithm with lazy fairness recompute and a bounded repair pass, built as a framework-agnostic TypeScript domain layer inside a NestJS/PostgreSQL application. The algorithm is in time, confirmed empirically by quadratic growth in the pairs scored across five-fold increases in scale and by reaching 97.6–100.0% of the optimal total against a min-cost max-flow baseline. The classical 1/2-approximation guarantee belongs to the static greedy step alone and does not extend to the deployed engine, whose quality is therefore established by measurement. Adding the fairness term moved Jain’s index by +0.0065 to +0.0288 across the contested 1.5:1 to 3:1 band and changed nothing at 10:1, where tutor capacity alone fixes the load vector; the change in average score that accompanies it is a weight-renormalisation artifact rather than a cost. The fairness tail was bounded exactly: the largest floor at which every student remains assignable exceeds the engine’s own worst placed pair by 0.0075 to 0.0241, and an exact assignment under that floor lifts the tail with no loss of total or coverage. Stability was measured rather than assumed: the engine leaves 4.93 to 42.03 blocking pairs per population where deferred acceptance leaves none, and the bounded stability pass built to reduce them removes 3.30 to 14.30 at no cost in coverage or total, but at a measured cost in load fairness and in the worst placed pair, so it stays opt-in. Every comparative claim is a mean over 30 independent populations with a 95% confidence interval and an exact paired sign test, and all six registered direction claims reproduced on a pre-registered held-out sample of seeds 1000–1029, with 23 of 24 effect sizes inside $\pm 50\%$ of their exploratory estimates.

Keywords: student–tutor matching; multi-criteria decision-making; weighted scoring; greedy assignment; fairness; explainability

Table of Contents

ABSTRACT.....	5
CHAPTER ONE.....	12
1.1 Background to the Study.....	12
1.2 Statement of Problem.....	13
1.3 Objectives of the Study.....	14
1.4 Research Questions.....	15
1.5 Scope of the Study.....	15
1.6 Limitations of the Study.....	16
1.7 Significance of the Study.....	17
1.8 Definition of Terms.....	17
CHAPTER TWO.....	19
2.1 Conceptual Framework.....	19
2.2 Theoretical Framework.....	19
2.3 Empirical Framework.....	20
2.3.1 Research Gap.....	22
2.4 Summary of Literature Review.....	22
CHAPTER THREE.....	23
3.1 Requirements Gathering.....	24
3.2 Developing an Algorithm to Match Students with Tutors Based on Established Criteria. .	28
3.3 Implementation Plan.....	32
CHAPTER FOUR.....	34
4.1 Analysis of Data Collection.....	34
4.1.1 Hardware and Software Stack.....	34

4.2 System Architecture.....	35
4.2.1 Three-Tier Architecture.....	36
4.2.2 Core Algorithm Architecture (Domain Layer).....	37
4.3 Database Design.....	39
4.3.1 Schema Design.....	39
4.4 Module Design.....	40
4.4.1 Backend Module Architecture.....	40
4.4.2 Consumption of the Matching Endpoint.....	42
4.5 Algorithm Design and Complexity Analysis.....	42
4.5.1 Composite Scoring Model.....	42
4.5.2 Priority-Queue Greedy Assignment Algorithm.....	44
4.5.3 Time Complexity Analysis.....	46
4.5.4 Space Complexity Analysis.....	48
4.5.5 Empirical Validation Against the True Optimum.....	48
4.6 Key Edge Cases and Their Handling.....	50
4.7 Implementation.....	51
4.7.1 Core Algorithm Implementation.....	51
4.7.2 Key Code Snippets.....	52
4.8 Module Implementation.....	54
4.8.1 Matchmaking Flow (End-to-End).....	54
4.9 Testing Strategy.....	56
4.9.1 Unit Testing Approach.....	56
4.9.2 Edge Case Verification.....	58
4.9.3 Test Execution.....	60
4.10 Results and Evaluation.....	60

4.10.1 Evaluation Harness.....	60
4.10.2 Match Quality and Fairness.....	60
4.10.3 Effect of the Fairness Weight.....	62
4.10.4 Performance Benchmarking.....	63
4.10.5 Comparison with Baseline Assignment Strategies.....	64
4.10.6 The Fairness Floor and the Worst-Placed Student.....	74
4.10.7 Stability of the Matching.....	69
4.10.8 Statistical Analysis of the Evaluation.....	71
4.10.9 Explainability.....	74
4.11 Discussion.....	75
CHAPTER FIVE.....	77
5.1 Summary of Findings.....	77
5.2 Conclusions.....	78
5.3 Recommendations.....	79
5.4 Contribution to Knowledge.....	80
5.5 Future Work.....	81
REFERENCES.....	83
APPENDIX.....	87
Appendix A: Source Code.....	87
Appendix B: Corrections Log (Seminar Proposal to Implemented System).....	87
Appendix C: Provenance of the Test Suite and Evaluation Figures.....	91

LIST OF TABLES

Table 1.1: Research questions.....	15
Table 1.2: Definition of terms.....	17
Table 2.1: Summary of closely related studies.....	21
Table 3.1: Requirements gathered and their sources.....	27
Table 3.2: Baseline criterion weights and the composite buckets they form.....	29
Table 4.1: Implementation technology stack (algorithm- and backend-relevant components).....	34
Table 4.2: Database tables in the matching workflow.....	39
Table 4.3: Backend feature modules and their endpoints.....	41
Table 4.4: Time complexity of each stage of the pipeline.....	46
Table 4.5: Space complexity by data structure.....	48
Table 4.6: Engine against the exact optimum by problem size.....	48
Table 4.7: Assignment approaches compared.....	49
Table 4.8: Edge cases and their handling.....	50
Table 4.9: Unit Test Coverage by Class (pnpm test:cov, 28 September 2026).....	56
Table 4.10: Edge Case Verification.....	59
Table 4.11: Match Quality and Fairness by Scenario (deployed engine; 30 independent populations per scenario; loadFactorWeight = 0.05).....	61
Table 4.12: Effect of the Fairness Weight by Ratio (deployed engine; 30 independent populations per scenario; $\delta = 0.05$ against $\delta = 0$).....	62
Table 4.13: Execution Time by Scale (stress-sweep, deployed engine, loadFactorWeight = 0.05; five runs per scale, 28 September 2026).....	63
Table 4.14: Baseline Comparison by Scenario (identical fixtures and scorer per scenario; 30 independent populations per arm; 28 September 2026).....	65

Table 4.15: The fairness floor and the worst-placed student (deployed engine vs the exact floor-constrained solve; 30 independent populations per scenario; 28 September 2026).....	67
Table 4.16: Blocking pairs per population by strategy (mean over 30 populations, exploratory seeds 0–29; 0 = a stable matching).....	69
Table 4.17: Cost of the stability pass (stability pass minus deployed engine; 30 populations; 28 September 2026).....	70
Table 4.18: Significance of the headline comparisons on the load-independent total per student (30 populations per scenario; W/L/T is the paired sign test against the deployed engine; 28 September 2026).....	72
Table 4.19: Held-out confirmation (seeds 1000–1029): exploratory value → held-out value for each registered direction claim.....	73
Table B.1: Changes between the Seminar Proposal and the Implemented System.....	87
Table C.1: Full evaluation sweep, reproduced verbatim (evaluation-results.csv).....	93
Table C.2: Per-run records for the timing rows (evaluation-per-run-results.csv).....	94
Table C.3: Engine against baseline arms, early capture run of 3 September 2026 (evaluation-capture-results.csv).....	94

LIST OF FIGURES

Figure 2.1: Conceptual framework of the matchmaking pipeline.....	19
Figure 3.1: Simplified flowchart of the criteria-based matching procedure (design overview)....	32
Figure 4.1: Use case diagram of the matchmaking system.....	36
Figure 4.2: Core UML package diagram.....	37
Figure 4.3: Class diagram of the core domain.....	38
Figure 4.4: CrowsFoot ERD.....	39
Figure 4.5: UML component diagram.....	41
Figure 4.6: Detailed flowchart of the priority-queue greedy assignment.....	46
Figure 4.7: Engine against the optimal assignment (share of the min-cost max-flow optimum, by problem size).....	49
Figure 4.8: Sequence diagram of the end-to-end matchmaking flow.....	54
Figure 4.9: Activity diagram of the end-to-end matchmaking flow.....	56
Figure 4.15: Core-layer test coverage output (pnpm test:cov, 28 September 2026).....	58
Figure 4.10: Fairness by ratio (Jain's index, engine with and without the fairness weight; 95% confidence intervals).....	63
Figure 4.11: Coverage by strategy (share of students placed; 30 populations per scenario).....	66
Figure 4.12: Placed-only average match score by strategy (30 populations per scenario; overlapping intervals are not distinguishable).....	67
Figure 4.13: Blocking pairs per student by strategy (0 = a stable matching; 30 populations per scenario).....	70
Figure 4.14: Each baseline against the engine on the load-independent total per student (mean per-population difference; 30 populations per scenario).....	73

CHAPTER ONE

INTRODUCTION

1.1 Background to the Study

Online learning has now become mainstream. The online tutoring services market alone, for instance, was estimated at USD 10.4 billion in 2024 and is forecast to almost double, to reach USD 23.7 billion in 2030 (Grand View Research, 2025), with students at the primary and secondary levels contributing over half of that demand. The increased popularity of learning services is due to the rising adoption of a broader trend in learning personalization – learning tailored to the unique needs and learning styles of each individual learner. This trend has been widely recognized in studies of educational recommender systems and is directly correlated with higher levels of engagement and better educational outcomes (Deschenes, 2020; da Silva et al., 2023; Spivakovsky et al., 2025).

The need for quality additional tutoring can be seen in the context of the West African secondary schools where the current research project is set up. According to WASSCE results of 2024, out of 1.8 million candidates, 72.12 percent scored credit passes in at least five subjects, including English Language and Mathematics, but this is a drop of 7.69 percentage points compared to the 2023 results (Vanguard, 2024). That left roughly half a million candidates without credit passes in at least five core subjects. Many others are working to improve their grades for competitive university admission, and hiring a private tutor for WAEC and NECO preparation has become the standard response. However, this process is still largely informal, with most people relying on recommendations from others and online sources.

Even where tutoring websites are available, the process of matching students with tutors is done using very primitive techniques such as manual assignment, keyword filtering, or even rules based on subject and availability only (Taveekarn et al., 2014; Ramesh, 2020). While this may be easy to implement, it does not consider various aspects that make up the suitability of a match, which includes subject expertise and proficiency level, tutor experience, language, schedule fit, teaching style compatibility, cost, and history of the tutor. Unsuccessful matches end up wasting hours of contact and even money for the paying family.

Besides this, there are issues of imbalances that occur within the platform since most of the tutors who are highly ranked have too many students to teach while the new tutors have none at all due to the ranking effect (Pitoura et al., 2022), and there are also clashes in schedules. Multi-criteria decision-making and combinatorial optimization provide an elegant solution to the two issues since they use the weighted scoring models to evaluate and solve conflicting constraints and even fairness constraints (Wang et al., 2023; Hien et al., 2025). This paper will use this approach to tackle the issue of matching of tutors and students in a system implemented using NestJS, React and PostgreSQL.

1.2 Statement of Problem

Online tutoring has grown into a large market, yet the core question of which tutor a student gets assigned to is still settled by methods the literature has already shown to be inadequate. Tutor-finder platforms match on subject and time slot and little else (Taveekarn et al., 2014). Some allocation algorithms bring in teaching style but leave scheduling and fairness out (Ramesh, 2020), and surveys of educational recommender systems note how rarely direct tutor allocation is attempted at all (da Silva et al., 2023). The situation gives rise to a set of concrete problems:

- *Limited matching criteria:* Current systems ignore important factors such as teaching style, language compatibility, pricing, and historical performance.
- *Scheduling conflicts.* A tutor's calendar and a student's calendar often fail to overlap, and the problem repeats week after week for anyone on a regular schedule.
- *Uneven tutor workload.* A few well-known tutors get flooded with requests while the rest sit idle, so demand piles up unevenly.
- *Rigid preference handling.* Most platforms apply the same criteria to every student and never adjust for personal priorities, such as how much price matters or a preference for experienced tutors.
- *Weak conflict handling.* Cancellations, reassignments, and two students competing for the same slot are dealt with poorly or not at all.

Every group involved feels the cost of these gaps. A student revising for an important exam can lose days to the search for a tutor. A family that takes on an unsuitable tutor ends up paying twice when the tutor has to be replaced, while tutors on the other side see fewer bookings simply because demand is spread unevenly. For the platform itself, an opaque matching process erodes users' trust in the recommendations it makes. The literature reviewed in Chapter Two turned up no study that brings multi-criteria decision-making, timetable feasibility, workload balancing, and explainable matching together in a single student-tutor allocation framework, and closing that gap is what this project sets out to do.

1.3 Objectives of the Study

General Objective

To build and assess a multi-criteria student-tutor matchmaking algorithm that improves compatibility, fairness, and scheduling efficiency in digital tutoring settings.

Specific Objectives

The study aimed to:

1. To define a weighted composite score for student-tutor compatibility that draws on subject expertise, grade level, experience, teaching style, price, and region, with each sub-score kept within $[0,1]$.
2. To give each student control over how much weight every matching criterion carries, renormalising the weights automatically after any change.
3. To develop a conflict-resolution mechanism for competing tutor requests and overlapping schedules, including deterministic tie-breaking and waitlisting.
4. To implement a fairness and load-balancing mechanism that distributes students equitably across tutors, and to identify the operating conditions under which it is effective.

5. To evaluate the algorithm's computational efficiency and scalability through formal complexity analysis and empirical benchmarking, and its match quality against an exact optimal baseline.
6. To integrate the algorithm into a full-stack web application demonstrating the end-to-end matchmaking workflow.

1.4 Research Questions

Table 1.1: Research questions

No.	Research Question
RQ 1	How can a multi-criteria weighted model be designed for student–tutor matching?
RQ 2	How can student preferences be dynamically incorporated into the matching process?
RQ 3	How can scheduling constraints and availability conflicts be resolved efficiently?
RQ 4	How can fairness and tutor load balancing be implemented in the system?
RQ 5	How can historical performance metrics improve match quality?
RQ 6	How scalable is the proposed algorithm compared to traditional matching methods?
RQ 7	How can explainability be integrated into the matchmaking process?

1.5 Scope of the Study

The current study considers the design, implementation and evaluation of a multi-criteria matchmaking system for secondary school tutoring that uses weighted compatibility scores, schedule matching and subject, grade level and examination type (WAEC, NECO etc.) preferences

filtering. Examination type is not used as a background notion but is implemented in the system explicitly. It is stored as an attribute of students and tutors (Section 4.3.1), taken into consideration at the eligibility filtering stage (Section 4.6) and affects the calculation of the academic compatibility score (Section 4.5.1). Evaluation of the system also involves realistic secondary school data from Nigeria (Section 4.10.1). The system also implements workload balancing and explains every match.

Technically, the study considers the implementation of the weighted scoring model, scheduling constraints handling, student-tutor conflict handling, and system evaluation for fairness, scalability and performance. The implementation of the system uses the three-tier architecture with the Next.js React frontend, NestJS backend where matchmaking algorithm is implemented in the domain layer of the system and PostgreSQL database managed via Drizzle ORM.

1.6 Limitations of the Study

- The system was analyzed using simulated data sets instead of actual large-scale data from the platform.
- Behavioral aspects of humans such as personality compatibility and emotional interaction are missing in the model.
- The performance analysis depends upon the nature of the feedback data, which may be biased or incomplete.
- Fairness here is defined by a fixed mathematical formula, which is not necessarily how users experience it in practice.
- The $1/2$ -approximation guarantee applies to the static greedy matching step alone, not to the deployed engine, which adds fairness to the priority function, top-K truncation, deterministic tie-breaks and a bounded repair pass; its solution quality is therefore established by measurement (97.6-100% of the exact optimum at the scales tested) rather than by that bound.

1.7 Significance of the Study

The work matters because it pairs algorithmic rigor with a genuine educational need. Matching stays fast and accurate, tutor loads remain balanced, and every tutor is assessed on objective data rather than reputation alone.

The study also adds to the theory of multi-criteria optimisation and explainable recommendation. Its complexity analysis, together with the finding that the fairness weight only helps within a specific load band, applies to matchmaking mechanisms in general, and to greedy priority-queue assignment with lazy fairness recomputation in particular.

1.8 Definition of Terms

Table 1.2: Definition of terms

Term	Definition
Student-Tutor Matchmaking System	An algorithmic system that assigns tutors to students based on multiple criteria.
Multi-Criteria Decision-Making (MCDM)	A method of evaluating options using multiple weighted factors.
Priority-Queue Greedy Assignment	An algorithm that processes the highest-scoring student-tutor pair first using a max-heap, with lazy fairness recomputation to catch stale scores.
Dynamic Weighting	Adjusting the importance of criteria based on user preferences.
Compatibility Score	A numerical value representing the suitability of a tutor for a student.
Hard Constraints	Conditions that must be strictly satisfied (e.g., subject match, grade-level and exam-type support, capacity check).
Soft Constraints	Flexible conditions that influence ranking but are not mandatory.

Load Balancing	Distribution of students among tutors to avoid overloading.
Fairness Score	A load-based score that decreases as a tutor approaches capacity, implemented as $\min(1, (1 - \text{LoadRatio}(t))^{1.15} + \text{cold-start boost})$ (Section 4.5.1).
EMA (Exponential Moving Average)	A weighted average that gives more importance to recent feedback, used for tutor rating updates.
Cold Start	The problem of scoring new tutors with no historical feedback; handled by a neutral default quality of 0.5.
Big-O Notation	A mathematical notation describing the upper bound of an algorithm's time or space complexity.

CHAPTER TWO

LITERATURE REVIEW

2.1 Conceptual Framework

Tutor matching has moved through three phases. It started as manual placement by an agency on subject and availability, then became rule-based software matching on subject and time slot (Taveekarn et al., 2014). The third phase adds teaching method, budget and tutor experience with some weighting, but still without a unified formula or a fairness system (Ramesh, 2020). This study belongs to that third phase, as the four-stage pipeline of Figure 2.1.

Figure 2.1: Conceptual framework of the matchmaking pipeline

Input, scoring and assignment feed forward; a feedback loop returns updated tutor quality to scoring.

Input collects the profile of both sides: subject required, academic level, availability, language, tutoring approach, expected payment and past ratings. Prior tutor-matching work treats these variables as both feasible and informative (Taveekarn et al., 2014; Ramesh, 2020).

Scoring turns every student-tutor pair into one normalised compatibility value. Four sub-scorers (AcademicScorer, PreferenceScorer, ScheduleScorer, FairnessScorer) each return a value in $[0, 1]$, and the CompositeScorer combines them through the weights α , β , γ and δ .

Assignment runs GreedyAssignmentEngine, which takes pairs from the highest score downward on a max-heap and recomputes the fairness term lazily so that it reflects the tutor's actual load. Feedback then updates each rating by exponential moving average with $\lambda = 0.3$.

2.2 Theoretical Framework

Tutor-student assignment is a multi-criteria decision problem: one tutor may know the subject better while another fits the timetable. Multi-criteria decision-making compares alternatives through normalisation, weighting and ranking (Wang et al., 2023; Hien et al., 2025), which makes a weighted compatibility score the right mechanism here instead of rule-based filtering.

Recommender research applies as well. Recommendations improve when user preferences, item properties and ranking sit in one coherent framework (Adomavicius & Tuzhilin, 2005; Burke, 2002), and the field has moved from algorithm development toward user-centred, context-aware systems (Konstan & Riedl, 2012; Ricci et al., 2022). Students are the users here and tutors are the items.

Recommendation quality also bears on learning. Educational recommenders have been applied to learning materials, peer recommendation and learning activities (Tarus et al., 2018; Deschenes, 2020; da Silva et al., 2023), but none recommends a personal tutor, and Karaci (2019) argues for structured learner modelling and feedback in intelligent tutoring systems.

Any recommendation must also be feasible. Time overlap, capacity and allocation order dominate scheduling (Leung, 2004; Pinedo, 2022), and feasibility alone is not enough: ranking can produce biased visibility (Pitoura et al., 2022), explanation improves trust (Zhang & Chen, 2020), and Coppolillo et al. (2024) show that the effects of a recommendation decay over time.

Greedy matching rests on bipartite matching theory: on any weighted bipartite graph, greedy returns at least half the optimal total score, by the exchange argument (Korte & Vygen, 2018; Preis, 1999).

2.3 Empirical Framework

The empirical work closest to this study falls into three streams, and they shape what this project inherited. The first is educational recommender systems, a mature body of work concerned with matching learners to materials, peers and activities rather than to tutors. Kester et al. (2007) showed early on that a matchmaking process built on competence, eligibility and availability can hold a learning community together, which is the same trio of criteria this system scores, but their community was one of learners and no tutor was ever allocated. The second stream is the small set of systems that assign tutors directly: FindMyTutor (Taveekarn et al., 2014) demonstrated that student-tutor matching can run end to end on a mobile client, yet its criteria stay narrow and neither fairness nor adaptive scoring is attempted, while Ramesh (2020) brought teaching and learning style into the selection and stopped short of scheduling. The third stream treats personalisation as an algorithmic problem in its own right: Lechuga and Doroudi (2023) group students from tutoring

data with methods richer than any tutor-finder uses, Spivakovsky et al. (2025) confirm that individualised support logic is what carries online tutoring, and Lotfian and Kabgani (2026) balance several objectives at once in a recommender that was never aimed at educational scheduling.

Read together, the studies point at the same hole from different sides. The recommender literature has the scoring machinery but no tutor at the end of it; the tutor-finding applications have the tutor but score on two or three criteria; the personalisation studies optimise for problems adjacent to this one. None of them offers a compact, weighted and explainable scoring framework for placing a student with a tutor, and that specific combination is what Table 2.1 summarises and what this project sets out to supply.

Table 2.1: Summary of closely related studies

Study	Focus	Strength	Limitation
Kester et al. (2007)	Learner matchmaking for knowledge sharing	Combines competence, eligibility, and availability	Focuses on learner communities, not tutor allocation
Taveekarn et al. (2014)	Mobile tutor-finding application	Implements direct student-tutor matching	Limited criteria, no fairness or adaptive scoring
Ramesh (2020)	Tutor matching by learning and teaching style	Includes style and level in tutor selection	Narrow scope, no scheduling optimization
Lechuga & Doroudi (2023)	Algorithmic grouping from tutoring data	Uses richer student data for grouping	Addresses peer support, not live tutor assignment
Spivakovsky et al. (2025)	Online tutoring for learning personalization	Confirms individualized support logic	No compact weighted scoring framework
Lotfian & Kabgani (2026)	Multi-objective recommender	Balances multiple objectives	Not designed for educational

	optimization	simultaneously	scheduling
--	--------------	----------------	------------

2.3.1 Research Gap

The literature still leaves a gap. Educational recommenders target learning materials, peer discovery or personalisation rather than tutor assignment for a given student, and the direct tutor-matching studies that exist rely on limited filters, with little regard for fairness, scheduling conflicts or explainability.

This study fills it by joining weighted scoring over four bounded sub-scorers (academic, preference, schedule, fairness) to a priority-queue greedy assignment with lazy fairness recompute and a feedback EMA update, in one implemented framework delivered as a full-stack application with the algorithm in a domain layer.

2.4 Summary of Literature Review

The chapter places student-tutor matching where three fields meet: multi-criteria decision-making, recommender systems and scheduling, with fairness and explainability now treated as design requirements rather than afterthoughts. Educational content and peer recommenders have advanced quickly, but student-tutor allocation remains largely unexplored and the engines in use depend on simple filters rather than a general weighting framework. The literature also supplies the design's mathematical basis, the proven $1/2$ -approximation of greedy bipartite matching, applied here directly rather than cited as background.

CHAPTER THREE

RESEARCH METHODOLOGY

Four methods could have run a project like this: SSADM, OOADM, Prototyping and Agile development. Each was considered before choosing one.

SSADM produces extensive documentation before any code and suits fixed, well-understood requirements, but adapts poorly when the design must change as implementation reveals new information, which happened repeatedly here (Appendix B records eighteen such changes between the seminar proposal and the final system). OOADM suits rich domain entities but says nothing about revising algorithmic components once coding exposes flaws. Prototyping gets early feedback on interfaces but offers no way to verify that an algorithm's mathematical properties, an approximation guarantee or a complexity bound, hold in the code.

The study therefore adopted design science (Hevner et al., 2004) run in short Agile iterations: the output is an artifact, an algorithm and the system that runs it, not an observation about an existing system. Each part of the specification (scoring, assignment, fairness, feedback) was designed, implemented, tested and corrected in a short cycle before the next began, instead of committing to one upfront design that could not be revised once its flaws surfaced.

Following the design science process logic of Peffers et al. (2007) and the staged practice recommended by Sommerville (2016), the work proceeded in six stages:

1. Problem identification and requirements gathering, drawing on the limitations of existing tutor-matching approaches identified in Chapter Two.
2. Selection of matching criteria and baseline weights for the composite scoring model.
3. System architecture and database design.
4. Algorithm specification, expressed as explicit equations with bounded output ranges (Section 4.5), so that each formula could be unit-tested independently.
5. Iterative implementation, in which each component was written, checked against the specification, and corrected whenever the code exposed a flaw in the original design, such

as an undefined edge case or a formula without a bound. A corrections log was kept through this stage (summarised in Appendix B) so that the reason behind every design change could be traced.

6. Evaluation, which covers unit tests, edge-case checks, and empirical benchmarking of the finished algorithm (Chapter Four).

Design science decided the research logic, and Agile decided how the design could change while the code was being written. A flaw found during implementation therefore cost one component's rewrite and not a chapter's, and the algorithm's formal properties (bounds, complexity, approximation guarantee) were written as testable requirements from the start.

3.1 Requirements Gathering

Requirements came from four places: the literature, the supervisor's review of the seminar proposal, a manual tracing of existing practice, and field sessions with students, lecturers and members of the public (Sommerville, 2016; Nuseibeh & Easterbrook, 2000). The literature was the set reviewed in Chapters One and Two, read for the criteria platforms actually apply and for where those criteria run out: FindMyTutor (Taveekarn et al., 2014), Ramesh's (2020) method, the Grand View Research (2025) market sizing and the WASSCE statistics (Vanguard, 2024). The supervisor's review of February 2026 fixed the scope, criteria and starting weights. Tracing the manual process by hand (Sections 1.1 and 2.1) is where the fairness and scheduling problems of Section 1.2 surfaced, and the research gap (Section 2.4) turned them into a specification. The sessions covered what no document could: what students and tutors here do when an arrangement is being agreed.

The sessions. Three groups used the system while the requirements were still open: 30 students of the Federal University of Technology Owerri, 5 lecturers, and 20 general users. All 55 worked on a running web interface rather than a drawing of one, which mattered, because the questions being asked were about how a profile form feels to fill in and how a ranked list reads at a glance. Each group fed a different part of the specification. The lecturers described what students and tutors look for when a placement is arranged. The general users said what they wanted to control themselves,

and complained about the platforms they had used. The students worked through the tutor-choice decision and explained what settled it.

The students. The literature could not answer the question the model depended on: with two tutors side by side, what decides the choice? The students said why they picked what they picked as they worked through the request flow. Price and distance from home counted for more than experience or rating with most of them, and availability was not a preference at all. A tutor whose free hours did not fit was out of the running rather than ranked lower, which is why schedule overlap became a hard constraint (FR4) rather than a score to be traded against, and why the weights sit with the student rather than being fixed once for the platform (FR3).

The lecturers. The five lecturers were asked what students and tutors look for in practice, and where the informal arrangement process goes wrong. Their answers produced the criteria in the composite score: subject first, then experience and the tutor's record, then language and teaching approach. They also made a point the students had no reason to notice, that a tutor's listed subjects mean little on their own, since a tutor who teaches a subject at one grade level or for one examination body may be no use for another. That is why eligibility on subject, grade level and examination type is settled before any score is computed (FR1), and why the criteria form the four bounded buckets of Section 3.2 (FR2) rather than a flat list.

The general users. The twenty general users were the harshest group and the most useful. Most had paid for tutoring. They named price, availability and the days and hours that work for them as the things they wanted to set themselves, and spent much of each session on what they disliked about the platforms and agencies they had tried. One complaint came up again and again: tutors with a pile of reviews stay at the top, and a tutor who has just joined never appears at all, so nobody books them and no reviews accumulate. The users did not put it in those terms, but the mechanism is the one that keeps established sellers in front of new ones in online marketplaces (Tadelis, 2016), and the one recommender research calls popularity bias, where exposure concentrates on the items that already hold it and the unfairness falls on the provider rather than the consumer (Pitoura et al., 2022; Deldjoo et al., 2024). Job and freelance platforms show the pattern at its clearest: an account with no work history is rarely given the job that would give it one.

Tutoring platforms inherit that structure, because they rank on accumulated rating and review count. Hence the load-balancing requirement (FR5), and the fairness term in the composite score. It is also why a tutor with no feedback scores a neutral 0.5 rather than zero (Section 4.5.1). The study does not claim to have solved the problem: spreading load protects the tutors already in the system, but nothing in the algorithm manufactures the visibility a review history buys, and fairness here is defined mathematically rather than as users experience it (Section 1.6).

Usability feedback. The sessions also tested the client, and what came back concerned the interface rather than the algorithm: how the pages felt to use, where a screen carried more than could be taken in, and whether the explanation behind a match was believed. That material shaped what the client shows by default and how much scoring detail a student sees before asking for more. It is recorded because it was collected, not because it changed the engine: no scoring function and no part of the assignment procedure was touched, and the client is outside the algorithmic scope of the study title (Section 4.1.1).

Consent. Participation was voluntary and unpaid, and the volunteers were students, lecturers and members of the public rather than a sample drawn for statistical purposes. Each of the 55 was told what the exercise involved, how long it would take and how the material would be used, and each gave consent for the session to be observed and for their behavioural responses and stated preferences to be recorded and used in the design. Anyone could stop at any point, and nothing was recorded beyond what a participant said and did in the session. No names, contact details, examination results or academic records were collected; notes were anonymised as they were written, only aggregate preferences reached the requirement list, and no real tutoring took place.

From these sources came the functional requirements: exclude tutors who cannot teach a student's subject before any ranking happens; score every remaining pair on several weighted criteria rather than subject and availability alone; respect each student's own preference weights; treat timetable feasibility as a hard constraint; spread students across tutors so highly rated ones are not swamped while newer tutors sit idle; fold post-session feedback into a tutor's standing; and give a reason for every match instead of an opaque ranking.

The academic aims produced the non-functional requirements: sub-scores bounded to a known range so the composite can be unit tested; an assignment fast enough for platform scale with a

verifiable complexity bound; reproducibility from a fixed random seed; and matching logic isolated from the web framework so it can be tested without HTTP or a database. Table 3.1 lists everything gathered at this stage.

Table 3.1: Requirements gathered and their sources

ID	Requirement	Type	Source
FR1	Exclude tutors who do not teach the student's subject, or support the grade level or exam type, before scoring	Functional	Ch. 2 gap; existing systems; lecturer sessions
FR2	Score each eligible pair on academic, preference, schedule, and fairness criteria	Functional	Ch. 2 gap; lecturer sessions
FR3	Apply each student's own preference weights to the criteria	Functional	Seminar proposal; student sessions
FR4	Treat schedule overlap as a hard feasibility constraint	Functional	Existing systems; student sessions
FR5	Balance tutor load so that no tutor is saturated while others are idle	Functional	Ch. 2 gap; user sessions
FR6	Update a tutor's rating from post-session feedback	Functional	Seminar proposal
FR7	Return an explanation (per-criterion breakdown) for every match	Functional	Ch. 2 gap
NFR1	Bound every sub-score to [0,1] so the composite is testable	Non-functional	Study aim
NFR2	Provide a stated, checkable time-complexity	Non-	Study aim

	bound	functional	
NFR3	Make evaluation reproducible from a fixed seed	Non-functional	Study aim
NFR4	Isolate the algorithm from the web framework for independent testing	Non-functional	Study aim
NFR5	Authenticate users and separate roles (student, tutor, admin)	Non-functional	Existing systems
NFR6	Collect no personal or academic records from participants; keep anonymised session notes only	Non-functional	Student, lecturer and user sessions

The elicitation also settled what data the system would consume. With no live dataset large enough (Section 1.6), a domain profile was built for the Nigerian secondary-school setting: subjects, grade levels, WAEC and NECO examination types, tutor experience and pricing, capacity and weekly availability. Its attributes come from the published material reviewed in Chapter Two and from the sessions above, and tutors preparing WAEC and NECO candidates went through the stored fields, fixing the ranges the generator works from. That profile drives the deterministic generator used throughout the evaluation (Section 4.10.1; Sections 4.10.2 to 4.10.4 analyse its datasets).

One point bounds what the study can claim. No real-world tutoring data was bought, shared or exchanged at any stage. No dataset was purchased, no student records were supplied by any school, tutoring business, platform or vendor, and nothing was traded for access to the sessions. The only human input to the design was the preferences and behaviour of the volunteers, recorded in the anonymised, aggregate form the requirement list uses, and every figure in the evaluation chapter was generated from the domain profile. Section 1.6 accordingly lists simulated data among the limitations: the results rest on that profile being a fair description of the setting rather than a sample of live users.

3.2 Developing an Algorithm to Match Students with Tutors Based on Established Criteria

With the requirements fixed, the second stage turned matching into a computable model. The criteria form four buckets, each yielding a sub-score bounded to $[0,1]$: academic fit, preference fit,

schedule fit and fairness. Every pair receives one composite score, the weighted sum of the four, with weights summing to one (Section 4.5).

Academic fit combines subject depth, graded only after eligibility has been enforced as a hard filter, with grade-level proximity and the tutor's experience and rating. Preference fit combines teaching-style similarity, budget compatibility and, in person, regional proximity. Schedule fit is the proportion of a student's requested slots that a single tutor slot covers. Fairness depends only on the tutor, favouring lighter loads so that assignment spreads. Baseline weights came from the requirements and a student may override them, after which they are re-normalised to one (Table 3.2).

Table 3.2: Baseline criterion weights and the composite buckets they form

Criterion (student-facing)	Weight	Composite bucket
Subject fit	0.30	Academic (α)
Experience	0.15	Academic (α)
Feedback / rating	0.10	Academic (α)
Language / style fit	0.15	Preference (β)
Availability	0.25	Schedule (γ)
Load factor	0.05	Fairness (δ)
Aggregated	$\alpha = 0.55, \beta = 0.15, \gamma = 0.25, \delta = 0.05$	Sum = 1.00

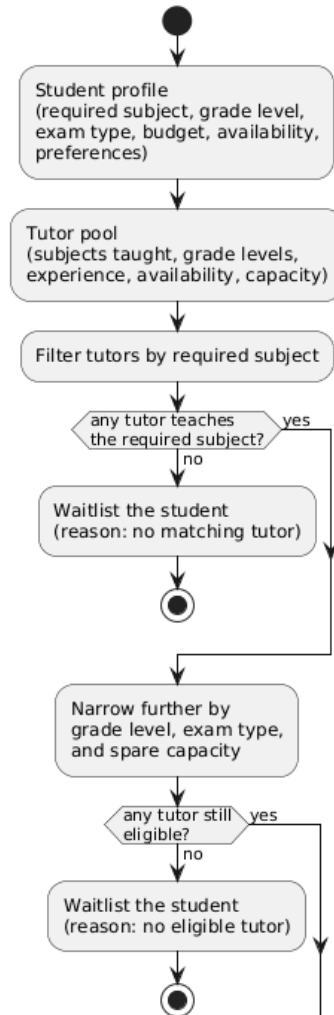
Each component was built in isolation, with an explicit output range and unit tests against hand-computed values, before integration. Assignment was cast as a bipartite b-matching solved by the priority-queue greedy engine with lazy fairness recompute, and a moving average updates each tutor's rating after every session (Section 4.5).

Algorithm 3.1: Criteria-based student–tutor matching (design overview)

7. For each student, discard tutors who are ineligible on subject, grade level, exam type, or spare capacity.
8. Score every surviving pair on the four bounded criteria and blend the results using the student's weights.
9. Push each eligible pair onto a max-heap ordered by its composite score.
10. Repeatedly take the best pair. If it is still valid, assign it and raise the tutor's load. If its fairness score has gone stale, re-push the pair with a fresh score instead of assigning it.
11. Put any student who remains unmatched on a waitlist, recording an explicit reason.
12. Once a session ends, update the tutor's rating and use it in later scoring.

Figure 3.1 compresses Algorithm 3.1: eligible pairs are scored on the four bounded criteria, pushed onto a max-heap and assigned from the highest score downward, each accepted pair raising the tutor's load before the next is considered (Section 4.5.2). It is laid out in two columns, read from the left column first.

How the Matching Algorithm Works (Simplified)



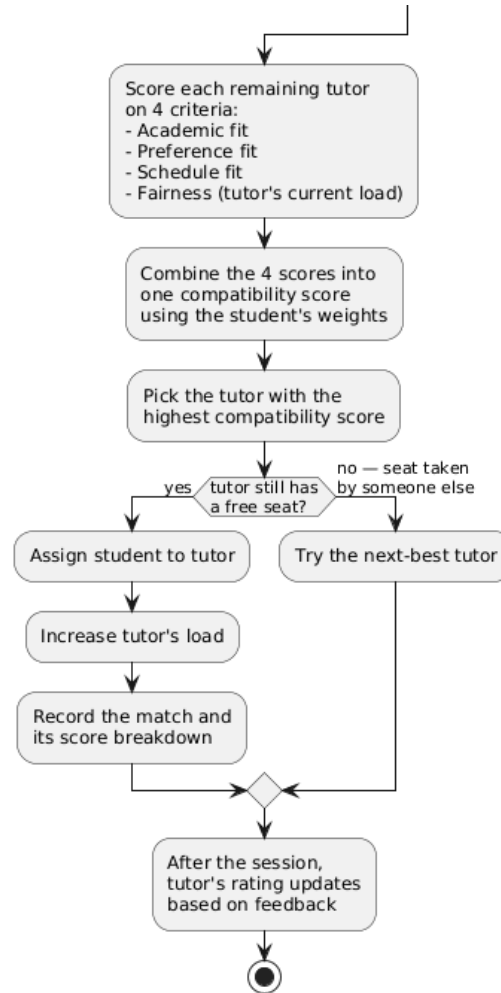


Figure 3.1: Simplified flowchart of the criteria-based matching procedure (design overview)

A second strategy was formalised for comparison: deferred acceptance, the student-proposing Gale-Shapley algorithm (Gale & Shapley, 1962; Roth, 2008) with tutor capacities, which yields a matching no pair would rather exchange. Stability and total score are different objectives, so it is a genuine competing baseline rather than a tuning variant; both run on identical fixtures and meet in Section 4.10.5 alongside the FCFS strategies platforms use.

3.3 Implementation Plan

The third stage decided how the model would be built and verified. The matching logic was implemented first as a framework-independent TypeScript layer, free of the web framework and the database, so that it could be tested against its specification before any HTTP or persistence code existed.

Implementation was planned in six phases:

13. Build the core domain layer (entities, the four scorers, the eligibility filter, the greedy assignment engine, and the feedback updater), each with unit tests asserting exact outputs.
14. Set up a standalone evaluation harness with a seeded synthetic data generator based on the domain profile compiled in Section 3.1.
15. Expose the core through NestJS feature modules (controllers, services, repositories) over a REST API.
16. Store state in PostgreSQL via Drizzle ORM, with migrations generated from the schema.
17. Build a Next.js client that exercises the workflow end to end.
18. Run the planned evaluation scenarios and write down the results.

The stack supports that ordering: NestJS for the application layer, TypeScript in strict mode for the core, Drizzle ORM over PostgreSQL for persistence, Jest for testing, and a pnpm-managed monorepo keeping backend and client as separate packages (Section 4.1.1).

The plan also fixed how the algorithm would be judged: five metrics tied to the research questions of Section 1.4, namely average compatibility of accepted matches (RQ1, RQ5), percentage of students unassigned (RQ3), Jain's fairness index (RQ4), wall-clock time across scales (RQ6) and the ratio of the greedy total to the exact min-cost max-flow optimum (RQ6), plus the static total per student (Section 4.10.5) and the worst-placed student (Section 4.10.6). Comparative results are means over 30 independent populations with a 95% confidence interval and a paired sign test, quoted only where they separate at $p < 0.05$; timings report minimum, mean and maximum over five runs per scale. Three baselines are measured against the engine (Section 4.10.5): filter-only FCFS, score-based self-selection and deferred acceptance (Gale-Shapley).

The datasets come from the harness generator, seeded so that every figure in Chapter Four is reproducible from the repository. Three scenario families were planned: a realistic 1:1 case mirroring the seed data, a moderate-load band (1.5:1 to 3:1 student:tutor ratios) that probes the fairness mechanism where capacity is slack, and a 10:1 stress sweep at four scales.

CHAPTER FOUR

SYSTEM DESIGN, IMPLEMENTATION AND RESULTS

4.1 Analysis of Data Collection

The data analysed in this chapter originate in the requirements stage (Section 3.1). With no live platform dataset of sufficient scale (Section 1.6), two kinds of data were used. The first is the domain profile: subjects, grade levels, WAEC and NECO examination types, tutor experience, pricing, capacity and weekly availability of the Nigerian secondary-school setting, drawn from Chapter Two and the seminar proposal. It fixes the attributes on which every generated student and tutor is built, so the results speak to the target population rather than to generic random inputs.

The second is the dataset that exercises the system: a deterministic 50-student, 50-tutor seed corpus drawn from that profile (the realistic-seed scenario), plus the synthetic families generated from the same distribution: moderate student:tutor ratios of 1.5:1 to 3:1, and a 10:1 stress sweep at four scales. Section 4.10.1 describes the generator and each family; Sections 4.10.2 to 4.10.5 report the analysis.

4.1.1 Hardware and Software Stack

Table 4.1: Implementation technology stack (algorithm- and backend-relevant components)

Layer	Technology	Version	Purpose
Backend framework	NestJS	11.x	Modular Node.js framework with decorator-based routing, dependency injection, and guards
Language	TypeScript	5.7	Strict-mode typed superset of JavaScript used for the backend and the core algorithm layer
ORM	Drizzle ORM	0.45	Type-safe SQL-first ORM with schema generation and migrations
Database	PostgreSQL	14+	ACID-compliant relational database

Authentication	Passport + JWT	—	Stateless authentication with access and refresh tokens
Testing	Jest	30.x	Unit and integration testing for the core algorithm layer

A browser-based client was also built to exercise the system end-to-end, but its implementation is outside the algorithmic scope stated in the study title and so is not treated as a subject of analysis here; only the parts of the workflow needed to demonstrate the matchmaking process (Section 4.2) are described.

4.2 System Architecture

Figure 4.1 presents the use case diagram: the actors who interact with the system (student, tutor, and administrator) and the principal functions each performs, from onboarding and requesting a match through to assignment and post-session feedback. How those use cases are realised is described in the rest of this section.

Figure 4.1 — Use Case Diagram of the Matchmaking System

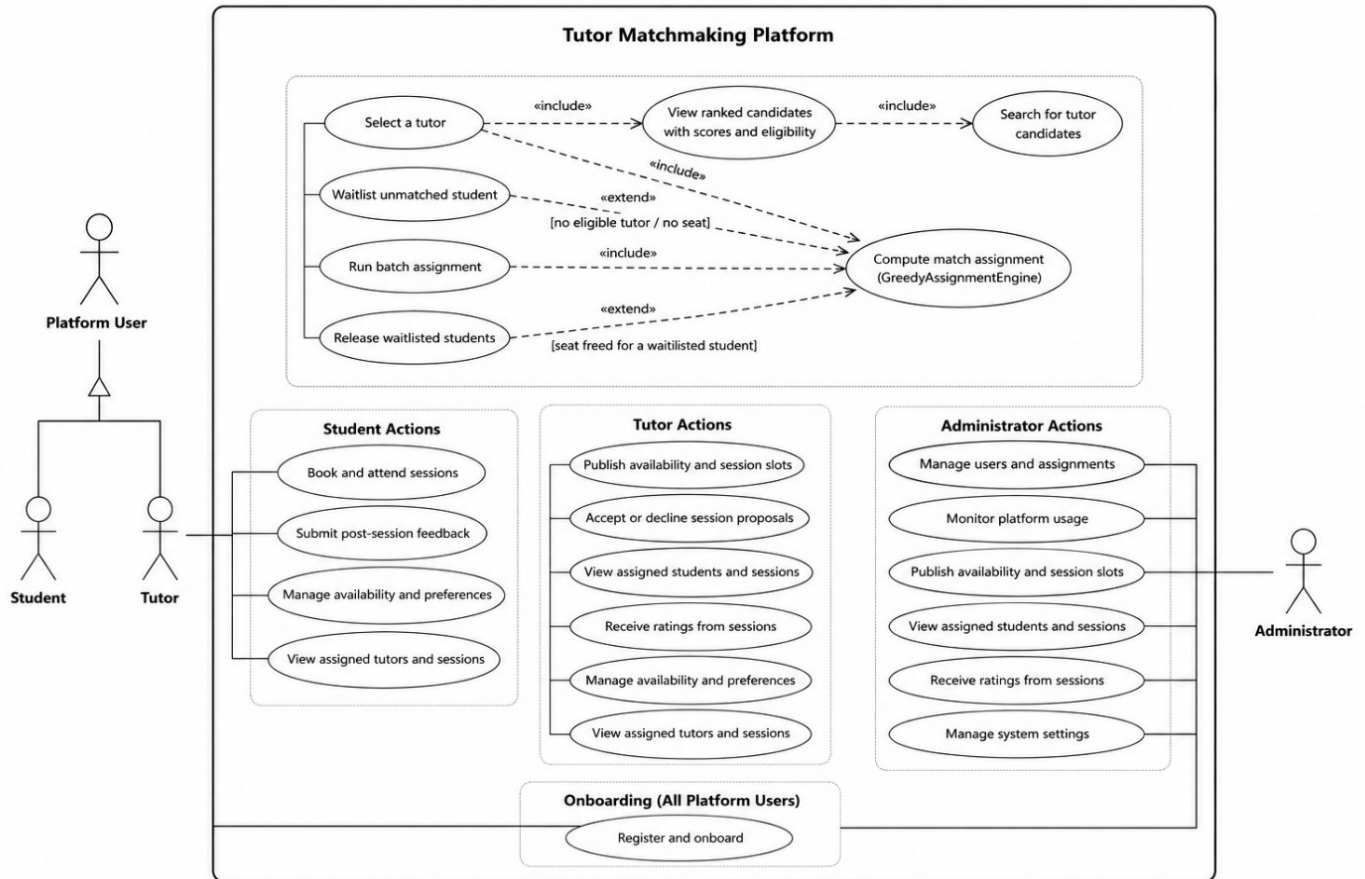


Figure 4.1: Use case diagram of the matchmaking system

4.2.1 Three-Tier Architecture

The system is organised as a three-tier architecture with strict separation of concerns:

- **Presentation Tier:** a Next.js React single-page application. Students, tutors, and administrators each get role-specific views, including dashboards, tutor discovery, session management, and profile editing. All communication with the backend happens over RESTful HTTP endpoints using Axios with automatic JWT token refresh.
- **Application Tier:** a NestJS server with ten feature modules — auth, users, courses, matchmaking, scheduling, sessions, messages, notifications, feed, and dashboard — wrapped around a standalone core algorithm layer. The core module is framework-independent; it contains the domain entities, value objects, scoring algorithms, and the

assignment engine. The matchmaking module links the core algorithms to the NestJS controllers and repository persistence.

- Data Tier: PostgreSQL holds twenty tables, of which the seven that carry the matching workflow are described in Section 4.3.1. Drizzle ORM supplies type-safe queries that are validated at compile time, and migrations are auto-generated from the schema definitions.

4.2.2 Core Algorithm Architecture (Domain Layer)

The core algorithm sits in a framework-independent domain layer under `src/core/`. Nothing in it touches NestJS, Express, the database, or HTTP, which makes it independently testable and reusable. The architecture follows clean architecture principles, with dependency inversion at the repository boundary.

Package structure shown in Figure 4.2.

Figure 4.2: Core UML package diagram

Each package is described briefly below

- *core/entities/*: holds the domain model (Student, Tutor, Assignment, and related value objects) along with their validation invariants.
- *core/algorithms/*: contains all scoring, filtering, ranking, and assignment logic, including the scorers, filters, ranker, assignment engine, feedback updater, and supporting data structures (MaxHeap, vector math).
- *core/engine/*: exposes the MatchingEngine facade, giving the application layer a single, unified entry point into the algorithms.
- *core/constants/*: centralizes tunable algorithm parameters (e.g. COLD_START_QUALITY, FEEDBACK_LAMBDA, LEVEL_MAX), each traceable back to the design specification.
- *core/enums/*: defines shared domain enumerations (AssignmentStatus, DeliveryMode, LearningStyle, etc.) used consistently across all layers.

- *core/interfaces/*: defines the repository interfaces (IStudentRepository, ITutorRepository, IAssignmentRepository) that the outer layer implements: the mechanism behind the dependency inversion shown in Figure 4.3.
- *core/evaluation/*: provides a standalone harness, including a synthetic data generator, for benchmarking and scalability measurement independent of the production database.
- *core/exceptions/*: defines domain-specific exceptions (IncompleteProfileException, NoEligibleTutorsException) with explicit, descriptive messages.

Where Figure 4.2 groups the domain into packages, Figure 4.3 gives the class-level design of the core: the principal entities (Student, Tutor, Assignment), the four scorers, the eligibility filter, the greedy assignment engine, and the feedback updater, together with the repository interfaces through which the outer layer supplies data: the relationship that realises the dependency inversion described above.

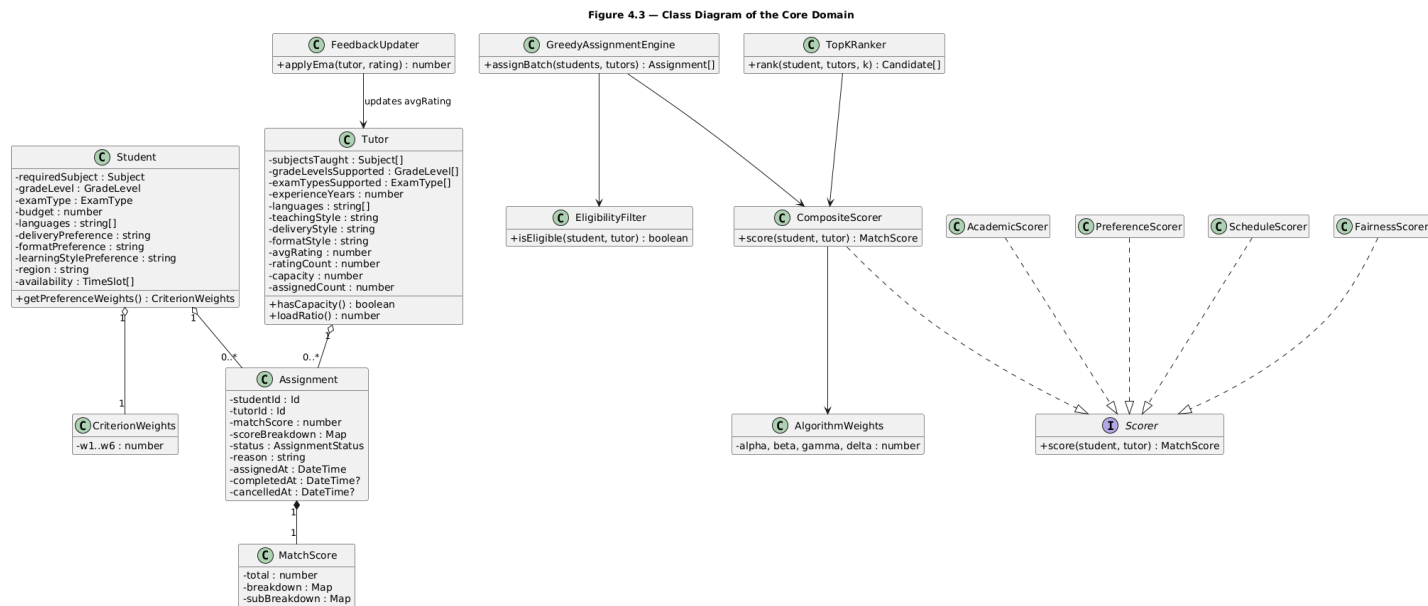


Figure 4.3: Class diagram of the core domain

4.3 Database Design

4.3.1 Schema Design

The schema is defined in `src/database/schema.ts` using Drizzle ORM's schema definition syntax, twenty tables organised around the user entity in a normalized design. Figure 4.4 shows the entity-relationship diagram for the seven tables in the matching workflow; the remaining thirteen support courses, the social feed, messaging and notifications, and are omitted here.

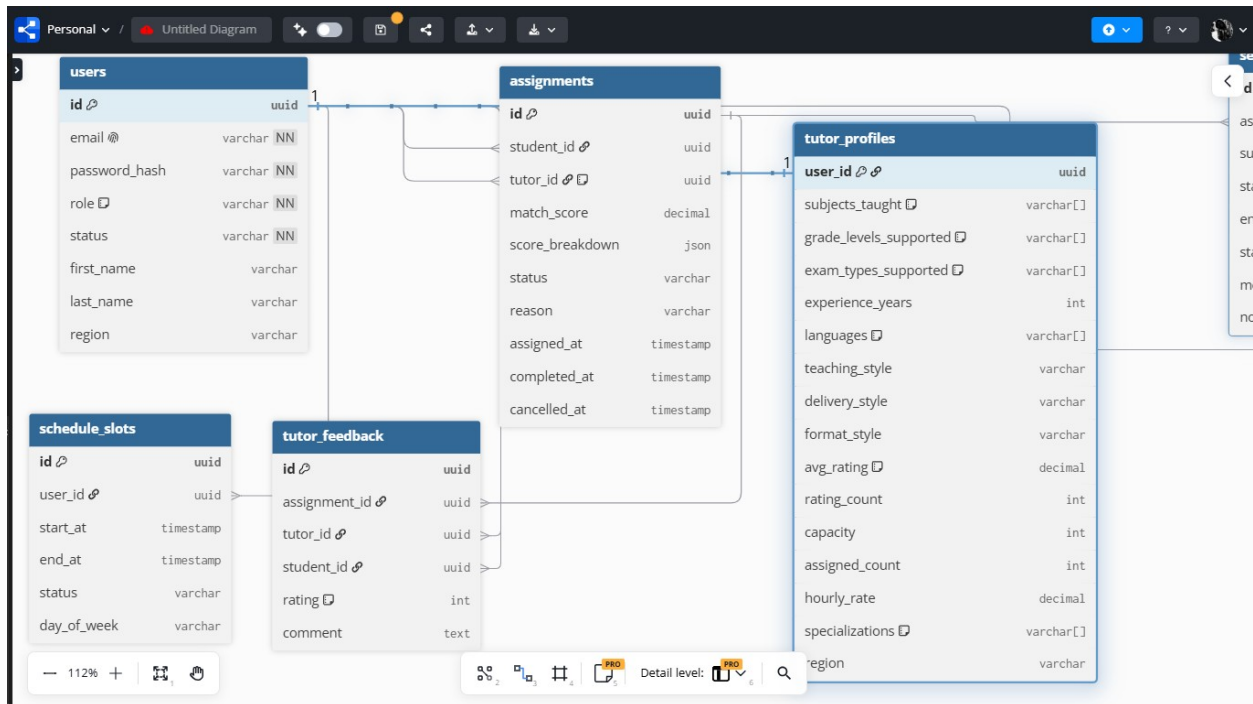


Figure 4.4: CrowsFoot ERD

As shown in Figure 4.4, users sits at the centre with `student_profiles` and `tutor_profiles` extending it one-to-one, keeping authentication fields (`email`, `passwordHash`, `role`, `status`) separate from role-specific matching data. `schedule_slots` records weekly availability; `assignments` is the central match record, referencing users twice and carrying the audit trail including `matchScore` and `scoreBreakdown`; `sessions` and `tutor_feedback` branch off it, feedback feeding the exponential moving average rating.

Table 4.2: Database tables in the matching workflow

Table	Key Columns	Purpose
-------	-------------	---------

users	id (uuid), email, passwordHash, role, status, firstName, lastName, region	Authentication and base identity
Student profiles	userId (FK), subjects[], requiredSubject, gradeLevel, examType, budget, languages[], deliveryPreference, formatPreference, learningStylePreference, region, preferenceWeights (JSON)	Student-specific profile and matching preferences
Tutor profiles	userId (FK), subjectsTaught[], gradeLevelsSupported[], examTypesSupported[], experienceYears, languages[], teachingStyle, deliveryStyle, formatStyle, avgRating (0-1 EMA), ratingCount, capacity, assignedCount, hourlyRate, specializations[], region	Tutor qualifications, availability, and load tracking
Schedule slots	id, userId (FK), startAt, endAt, status, dayOfWeek	Weekly availability schedules for students and tutors
assignments	id, studentId (FK), tutorId (FK, nullable), matchScore, scoreBreakdown (JSON), status, reason, assignedAt, completedAt, cancelledAt	Match records with full audit trail
sessions	id, assignmentId (FK), subject, startAt, endAt, status, meetingUrl, notes	Scheduled tutoring sessions
Tutor feedback	id, assignmentId (FK), tutorId (FK), studentId (FK), rating (1-5), comment	Per-session feedback for EMA rating updates

4.4 Module Design

4.4.1 Backend Module Architecture

The NestJS backend comprises ten feature modules, each following the controller–service–repository pattern. Figure 4.5 shows the five that participate in the matchmaking workflow and

their dependencies on one another and on the core domain layer; the remaining modules (messaging, notifications, dashboard analytics, the social feed) provide standard platform functionality with no bearing on the algorithm and are not analysed further, consistent with the scope

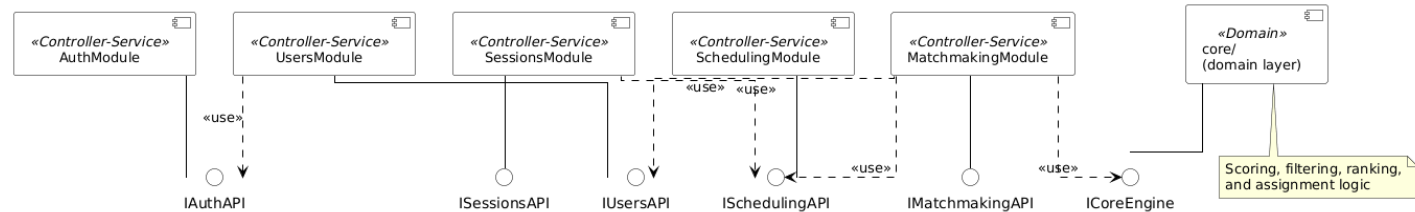


Figure 4.5: UML component diagram

Each module exposes a provided interface and depends only on the interfaces it needs. MatchmakingModule is the hub, calling UsersModule for profiles, SchedulingModule for availability and the core/ layer (Section 4.2.2) for entities and ranking; SessionsModule checks availability before booking; UsersModule relies on AuthModule for identity. Algorithm-facing logic stays in MatchmakingModule and core/; everything else is a supporting service.

Table 4.3: Backend feature modules and their endpoints

Module	Controller Endpoints	Responsibilities
Auth Module	/auth/signup, /auth/login, /auth/onboard, /auth/verify, /auth/logout, /auth/refresh	Registration, JWT issuance, role-based onboarding, token refresh
Users Module	/users/:id, /users/me, /users/me/student-preferences, /users/me/tutor-preferences	Profile CRUD and preference management
Matchmaking Module	/matchmaking/candidates, /matchmaking/candidates/students, /matchmaking/select, /matchmaking/batch, /matchmaking/assignments/me, /matchmaking/assignments/:id/status,	Scoring, ranking, assignment, feedback ingestion (bridges to core algorithms)

	/matchmaking/assignments/:id/feedback	
Scheduling Module	/schedules/tutors/:tutorId/slots, /schedules/tutors/:tutorId/sessions	Availability management and time slot queries
Sessions Module	/sessions, /sessions/me, /sessions/:id/accept, /sessions/:id/decline, /sessions/:id/propose	Session booking, lifecycle, and time proposal

4.4.2 Consumption of the Matching Endpoint

A single read endpoint, GET /matchmaking/candidates, reaches the engine: the matchmaking service pulls the student's profile, every active tutor profile and their slots, builds the Section 4.2.2 entities, and hands them to the TopKRanker. The response carries ranked candidates with match score, eligibility and score breakdown, shown as a percentage with a per-criterion explanation (Section 4.10.9). General client concerns fall outside the scope of Section 1.5.

4.5 Algorithm Design and Complexity Analysis

4.5.1 Composite Scoring Model

Everything in the matchmaking system depends on a composite scoring function that gives each student-tutor pair a compatibility score. The scoring function is organised around four buckets:

Equation 4.1: Top-Level Compatibility Score

Where $\alpha + \beta + \gamma + \delta = 1$, and each sub-score is bounded to [0,1].

Equation 4.2: Academic Score A

Where for an exact subject-plus-specialization match, 0.7 when the subject matches but the stated specializations do not align, and 0.5 when the subject matches but specialization data is missing. when the tutor supports the student's target exam type (e.g. WAEC or NECO), or when either side

leaves it unspecified. An explicit mismatch scores 0.5, so an exam-type clash softens subject depth without ruling out an otherwise capable tutor. where g is the gap between the student's grade level and the nearest grade level the tutor supports, and .

Equation 4.3: Preference Score P

Style uses cosine similarity on one-hot encoded vectors for delivery mode, format preference and learning style. Budget returns 1 if the tutor's hourly rate is within the student's budget, with linear decay for excesses. Region returns 1 for online sessions and in-person matches in the same region, 0 for an in-person region mismatch, and a 0.5 neutral default when region data is missing.

Equation 4.4: Schedule Score S

Here H_s and H_t are the student's requested and the tutor's available time slots. For a slot to count, a single contiguous tutor slot must cover it; fragments split across several tutor slots fail the contiguous-slot requirement. A student with zero availability slots is rejected as an incomplete profile.

Equation 4.5: Fairness Score $F(t)$

The exponent 1.15 makes the penalty on heavily loaded tutors slightly more than linear (Appendix B); $b(t)$ is a cold-start boost of 0.05 that applies only when a tutor has zero assigned students, nudging unused tutors into circulation. By default the load factor weight δ is 0.05.

4.5.2 Priority-Queue Greedy Assignment Algorithm

The assignment algorithm is a max-heap priority queue with lazy fairness recomputation, which fixes the staleness of static sorting. Eligibility (Algorithm 4.1) is a hard pre-filter: the tutor must teach one of the student's subjects, support the grade level and exam type, and have spare capacity; ineligible pairs are never scored. The heap alone is incomplete, since it never revisits a decision and can leave a student unplaced while a seat is held by someone with an acceptable alternative. Once the heap drains, a bounded repair pass (Algorithm 4.1, step 4) searches for each unplaced student a depth-bounded augmenting path that re-routes seated students, applies the deepest move first, and accepts it only when the number of placements strictly rises. The search is deterministic (static-score order, tutor id tie-break), bounded by a depth of three displacements plus a work budget, and returns immediately if no tutor holds a free seat.

Algorithm 4.1: Priority-Queue Greedy with Lazy Fairness Recompute

Input: Students list S, Tutors list T

Output: Assignments A, Unassignable list U

1. Initialize empty max-heap H

2. For each student s in S:

3. While H is not empty:

4. Bounded repair pass: for each student still unplaced, search for a depth-bounded augmenting path that re-routes already-seated students to tutors with spare capacity, and apply it when it strictly increases the number of placements
5. Students still not assigned are marked unassignable with reason

Figure 4.6 traces the loop in full, including the lazy recomputation that re-pushes a stale pair with a fresh score; Figure 3.1 gave the simplified overview. The steps of Algorithm 4.1 map onto the boxes of the figure, with step 4, the bounded repair pass, running after the loop rather than inside it. The two columns read from the left first.

Algorithm 4.1 — Priority-Queue Greedy Assignment with Lazy Fairness Recompute

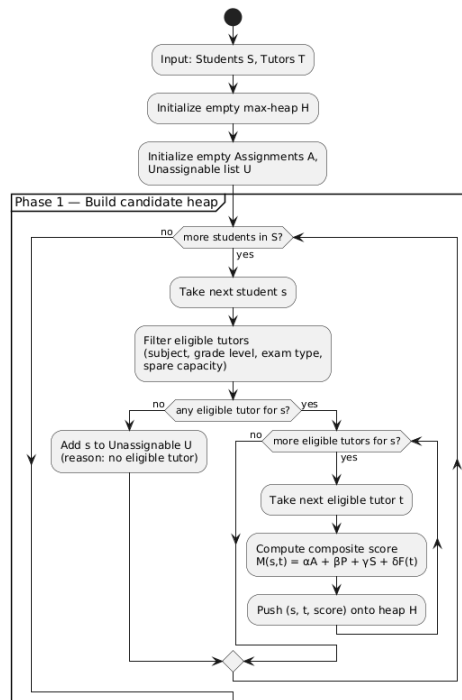


Figure 4.6: Detailed flowchart of the priority-queue greedy assignment

		count”)
Total (batch)		For S proportional to T proportional to n, with bounded profile sizes and bounded tutor capacity; the n^2 term comes from exhaustive pair scoring, the log factor from the sort/heap machinery
Top-K ranking (per student, TopKRanker)		Full sort then slice; adequate at current platform scale
Feedback update		Single EMA computation per feedback submission
Weight adaptation		Fixed 4 keys, proportional renormalization

This bound was verified against the running system. At five-fold increases in scale (200 to 1,000 to 5,000 students), pairs scored grew from 216 to 6,769 to 176,790, a 31-fold then 26-fold increase, close to the quadratic prediction. Mean elapsed time grew roughly 17-fold then 19-fold, quadratic with a slowly growing log factor, consistent with the claim (Section 4.10.4).

Two implementation-level optimisations reduce the constant factor on this bound without changing its asymptotic class:

- **Top-K truncation (topK: k)** caps heap memory at instead of , with a fallback pass of over any truncation-stranded students (U = the unassigned tail), so no student is silently dropped by the truncation itself.
- **Subject indexing (useSubjectIndex)** swaps T for T_{subject} , the number of tutors teaching a given student’s subject, in the scoring phase. It only helps when subject overlap is sparse relative to the full tutor pool.

Neither optimisation changes the pair-scoring floor; only candidate pruning at the eligibility stage can do that.

A single-student call (assignIncremental) runs assignBatch with $S = 1$, giving ; it matches one newly registered student against the current pool without a full batch re-run. It is not a streaming algorithm and carries no $1/2$ guarantee, which depends on a global greedy choice across all students.

4.5.4 Space Complexity Analysis

Table 4.5: Space complexity by data structure

Data Structure	Space	Notes
Max-heap (batch mode)	or	Full mode holds one entry per eligible pair; top-K mode caps this at
Assigned student set		Tracks which students have been assigned
Tutor capacity map		In-place mutation of tutor objects
Score breakdowns		Per-assignment score breakdown for explainability
Single-student request (assignIncremental)		Holds only that one student's candidate set in memory, not the full $S \times T$ structure

4.5.5 Empirical Validation Against the True Optimum

The classical exchange argument for greedy weighted bipartite matching gives a $1/2$ worst-case guarantee on total score (Preis, 1999; Korte & Vygen, 2018), but only for the pure greedy step, which is what steps 1 to 3 of Algorithm 4.1 implement. Four implementation choices break the proof's assumptions: the fairness term in the priority function, top-k truncation, hash tie-breaks, and the repair pass that re-routes seated students. The $1/2$ bound therefore describes the static, no-repair, untruncated variant, and the deployed engine is judged by measurement.

The deployed engine (heap pass plus repair) was checked against a true optimal baseline computed by an exact min-cost max-flow solver (optimal-baseline.ts), run only at small sizes (up to 100 students) because its own worst-case cost is approximately :

Table 4.6: Engine against the exact optimum by problem size

Size	Greedy Assigned	Optimal Assigned	Score Ratio
10	2	2	1.0000
25	10	10	0.9992
50	25	25	0.9998

100	63	64	0.9761

At the three smallest sizes the engine matched the optimum's placement count and 99.9-100.0% of its total score; at the largest (100 students, 33 tutors) it placed 63 of 64 and reached 97.6%. The solver's cost grows as approximately : 7.9 s at 1,000 students and 63.8 s at 2,000 on the same hardware (Section 4.10.4). The trade at 100 students is close (16 ms against 13 ms), empirical case for the engine, not the flow solver, in production.

Figure 4.7 plots the same comparison: the share of the min-cost-max-flow optimum the engine reaches at each size.

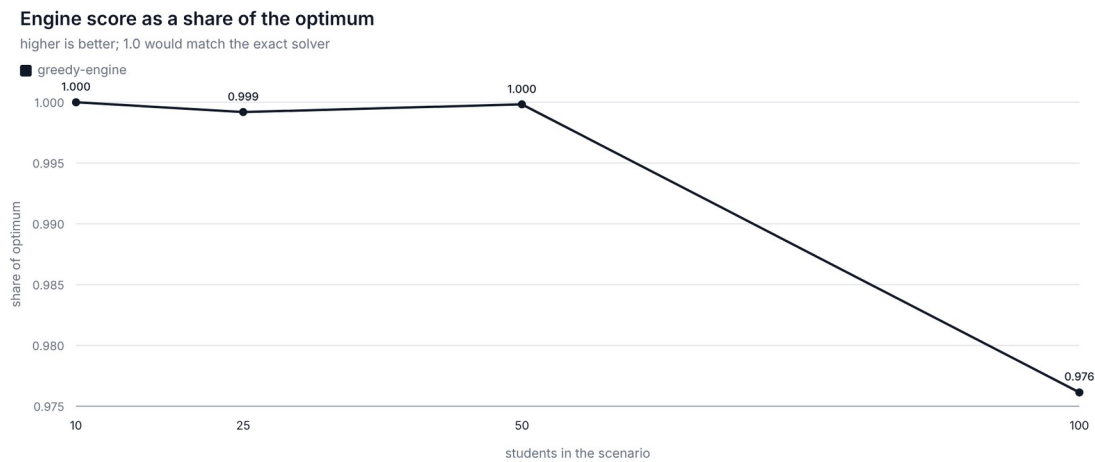


Figure 4.7: Engine against the optimal assignment (share of the min-cost max-flow optimum, by problem size)

Table 4.7: Assignment approaches compared

Approach	Time Complexity	Optimality	Trade-off
Greedy (this study)		Proven $\geq 1/2$ for the static no-repair variant; measured 97.6-100% of optimal for the deployed	Fast, simple, explainable; used in production

		engine at tested scales	
Min-cost max-flow (evaluation only)		Global optimum (exact)	Used only as an offline evaluation baseline, not in the production path
Hungarian Algorithm		Global optimum	Impractical for large n; batch-only
Genetic Algorithm		Approximate global	Non-deterministic; conflicts with explainability

4.6 Key Edge Cases and Their Handling

Design and implementation surfaced twelve edge cases, traced in the corrections log, Appendix B, each handled explicitly with a documented strategy. One further case is structural rather than input-driven: a student can be left unplaced while a seat is held by someone who has an acceptable alternative. Algorithm 4.1 step 4, the bounded repair pass of Section 4.5.2, handles it.

Table 4.8: Edge cases and their handling

Edge Case	Handling Strategy
Zero eligible tutors after hard filter	Returns explicit <code>NoEligibleTutorsException</code> with student ID and subject; student gets waitlisted assignment with reason
Tutor at full capacity (assignedCount = capacity)	<code>EligibilityFilter.hasCapacity()</code> returns false; tutor is excluded upfront and cannot be selected
Tutor with capacity set to zero	Caught by the same capacity gate; the tutor is removed before scoring and can never be selected
Student preference weights don't sum to 1	Auto-normalization: divides each weight by the sum of all weights (preserves relative priorities)
Student submitted zero availability slots	<code>IncompleteProfileException</code> thrown; student excluded from matching until availability is set
Availability request split across non-contiguous tutor	<code>ScheduleScorer</code> grants credit only when a requested slot is covered by a single contiguous tutor slot; fragmented

slots	coverage earns partial credit, never full
Tie in MatchScore between two or more tutors	Tiebreaker: lower assignedCount first, then lower tutor id, then a deterministic FNV-1a hash for reproducible output
New tutor with avgRating = null	Cold start default of 0.5 applied; prevents permanent burying of new tutors
Tutor slot filled mid-batch	Capacity is rechecked when a pair is popped from the heap; stale entries are skipped or re-pushed with a fresh key (lazy fairness recompute)
Incremental request arrives for tutor at capacity	Student is waitlisted rather than receiving an error or being silently dropped
Assignment is cancelled	Tutor's assignedCount is decremented; waitlist is rechecked for potential promotion of a waitlisted student
Oversubscription (demand exceeds aggregate capacity)	Students beyond aggregate capacity are returned as unassignable, each with an explicit reason

4.7 Implementation

Implementation used a pnpm monorepo with two packages, ``backend/`` (NestJS) and ``frontend/`` (Next.js). The core algorithm layer sits entirely inside the backend package, yet it is fully decoupled from framework concerns.

4.7.1 Core Algorithm Implementation

The core algorithm lives in a TypeScript domain layer under ``backend/src/core/``. The key implementation details are:

Scoring Service (Composite Scorer):

The Composite Scorer coordinates four independent sub-scorers. The Academic Scorer covers three sub-criteria: subject depth, level compatibility and experience quality. The Preference Scorer

applies cosine similarity to one-hot encoded style vectors. The Schedule Scorer measures availability overlap with a contiguous-slot requirement. The Fairness Scorer derives load ratio with a cold-start boost.

Assignment Engine (Greedy Assignment Engine):

The engine realises the priority-queue greedy algorithm with a custom MaxHeap. Its priority function blends the static score with the dynamic fairness score ($\delta * F(t)$); a deterministic FNV-1a tie-break keeps the output reproducible; and lazy fairness recomputation discards stale heap entries on pop. Once the heap drains, the same engine runs the bounded repair pass (Algorithm 4.1, step 4): a deterministic, depth-bounded augmenting-path search that seats a student the heap pass leaked by re-routing a seated student, accepting only moves that strictly increase the placement count and returning immediately when no tutor has a free seat. Repair is the engine's default behaviour, not an optional extra; a flag exists only to disable it for ablation, and every measurement in Section 4.10 was taken with it on.

Four representative excerpts in Section 4.7.2 illustrate these components directly; the full source code is available in the project repository referenced in Appendix A.

4.7.2 Key Code Snippets

Snippet 4.1: CompositeScorer.score()

```
score(student, tutor) {  
  const weights = AlgorithmWeights.from(  
    CriterionWeights.from(student.preferenceWeights)  
  );  
  
  const academic = this.academicScorer.score(student, tutor, weights);  
  const preference = this.preferenceScorer.score(student, tutor, weights);  
  const schedule = this.scheduleScorer.score(student, tutor);  
  const fairness = this.fairnessScorer.score(tutor);  
  
  const total =  
    weights.alpha * academic.total +  
    weights.beta * preference.total +  
    weights.gamma * schedule +  
    weights.delta * fairness;  
  
  return new MatchScore(total, breakdown, subBreakdown);  
}
```

Snippet 4.2: GreedyAssignmentEngine.assignBatch()

```
assignBatch(students, tutors) {
  const heap = new MaxHeap();

  for (const student of students) {
    for (const tutor of tutors) {
      if (!this.eligibilityFilter.isEligible(student, tutor)) continue;

      const score = this.compositeScorer.staticScore(student, tutor);
      heap.push({ student, tutor, score }, this.priority(score, tutor));
    }
  }

  while (heap.size > 0) {
    const item = heap.pop();

    if (this.canAssign(item.student, item.tutor)) {
      assignments.push(item);
      item.tutor.assignedCount++;
    }
  }

  return { assignments, unassignable };
}
```

Snippet 4.3: FairnessScorer.score()

```
score(tutor) {
  if (tutor.assignedCount >= tutor.capacity) {
    return 0;
  }

  const ratio = 1 - tutor.assignedCount / tutor.capacity;
  const boost = tutor.assignedCount === 0 ? 0.05 : 0;

  return Math.min(1, Math.pow(ratio, 1.15) + boost);
}
```

Snippet 4.4: FeedbackUpdater.updateQuality()

```
updateQuality(currentQuality, rating) {
  const oldQuality = currentQuality ?? COLD_START_QUALITY;
  const feedback = rating / MAX_RATING;

  return (
    (1 - FEEDBACK_LAMBDA) * oldQuality +
    FEEDBACK_LAMBDA * feedback
  );
}
```

```

);
}

```

4.8 Module Implementation

4.8.1 Matchmaking Flow (End-to-End)

The end-to-end matchmaking flow works as follows:

Figure 4.8 shows the same flow as a sequence diagram, tracing a single request from the student through the controller, service, core algorithm, and repository layers and back; the numbered steps below describe the same sequence in words.

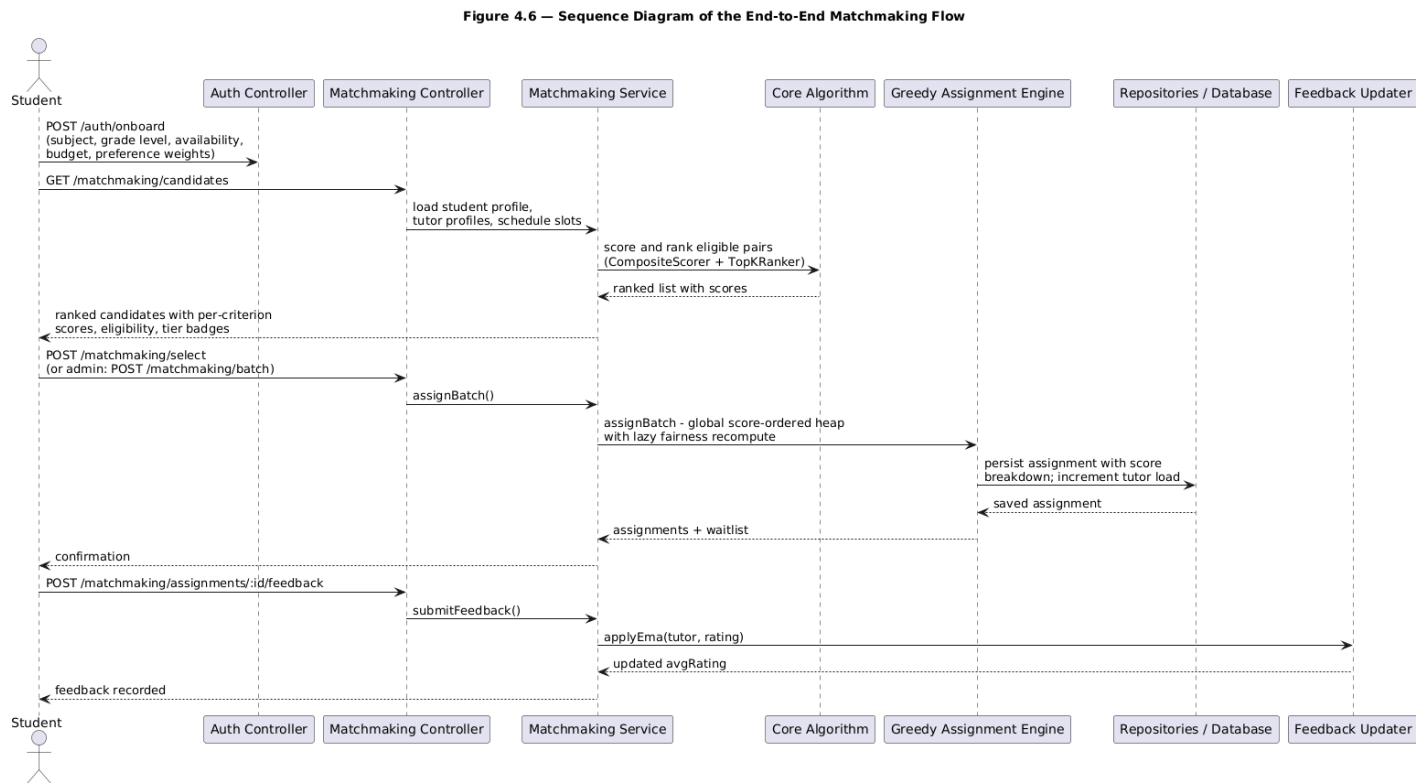
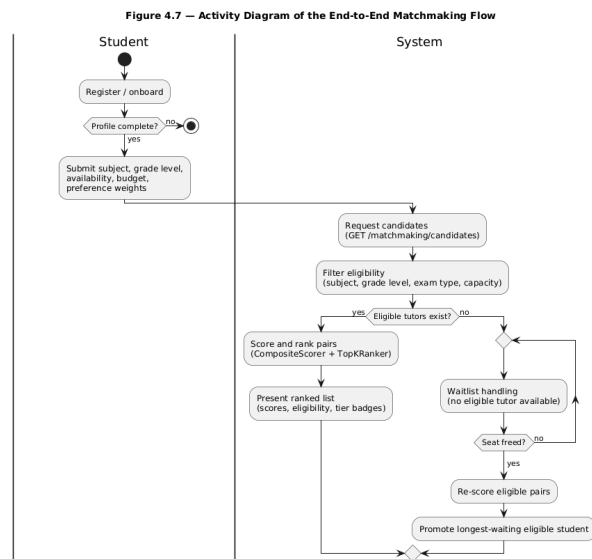


Figure 4.8: Sequence diagram of the end-to-end matchmaking flow

1. The student registers and onboard via `POST /auth/onboard`, supplying the subject, grade level, availability, budget, and preference weights.
2. To find tutor candidates, the student calls `GET /matchmaking/candidates`. The backend reads the student profile, every tutor profile, and the schedule slots.

3. TopKRanker ranks all possible student-tutor pairs using the CompositeScorer, producing a ranking of tutors with a score per criterion.
4. The front end presents the ranking of tutors with score percentages (0-100), eligibility, and tier badges for the top matches.
5. The student then chooses a tutor via `POST /matchmaking/select`, or the admin triggers `POST /matchmaking/batch`.
6. GreedyAssignmentEngine makes the assignment, increments the assignedCount, and saves the match with its per-criterion score.
7. After the session, the student calls `POST /matchmaking/assignments/:id/feedback`, and the FeedbackUpdater applies EMA to update the tutor's avgRating.

Figure 4.9 redraws the same flow as an activity diagram, read from the left column first. The diagram makes the decision points explicit: profile completeness, the presence of eligible tutors, and seat availability during assignment.



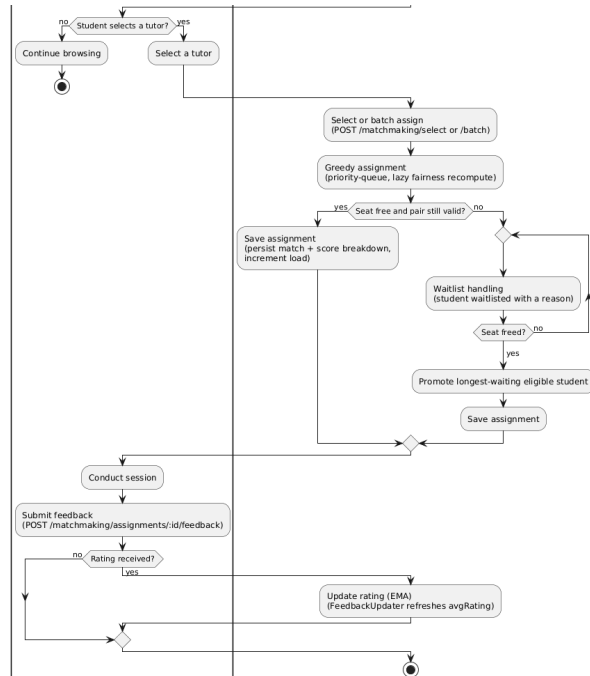


Figure 4.9: Activity diagram of the end-to-end matchmaking flow

4.9 Testing Strategy

4.9.1 Unit Testing Approach

Unit tests target the core algorithm layer in isolation (Jest 30, nine spec files, 197 tests). Two predate the evaluation stages: `core-engine.spec.ts` (23 integration-style tests, including top-K truncation behaviour) and `core-units.spec.ts` (55 focused tests closing the coverage gaps found during evaluation); both run independently of NestJS, consistent with Section 4.2.2. The remaining seven are the evaluation-stage suites: the harness (22), the evaluation TUI (49), the statistics helpers (18), and the stage-1 to stage-4 suites (12, 6, 6, 6) pinning the CSV contract, the oracle gap, the solver exactness against brute force, and the stability pass invariants.

Table 4.9: Unit Test Coverage by Class (pnpm test:cov, 28 September 2026)

Class	Statement Coverage	Branch Coverage	Key Coverage
CompositeScorer	100.00%	100.00%	Weight normalization, score range

FairnessScorer	100.00%	100.00%	Load ratio, cold-start boost, capacity
MaxHeap	100.00%	100.00%	Push/pop ordering, tie-breaking
VectorMath	100.00%	87.50%	Cosine similarity edge cases
GreedyAssignmentEngine	92.29%	83.08%	Batch, incremental, lazy recompute, bounded repair pass, top-K truncation
PreferenceScorer	96.55%	81.81%	Style similarity, budget, region
ScheduleScorer	100.00%	100.00%	Slot overlap, contiguous requirement
AcademicScorer	96.29%	88.88%	Subject depth, exam-type fit, level clamping, cold start
FeedbackUpdater	85.71%	85.71%	EMA update, cold start
EligibilityFilter	82.14%	83.33%	Subject, grade-level and exam-type gating, capacity
WeightAdaptation	100.00%	50.00%	Proportional renormalization after a weight bump
AssignmentLifecycle	85.71%	77.77%	Waitlist release on completion or cancellation

C:\WINDOWS\system32\cmd.exe					
index.ts	0	100	100	0	1-6
src/core/algorithms	100	100	100	100	
index.ts	100	100	100	100	
src/core/algorithms/adaptation	100	50	100	100	
weight-adaptation.ts	100	50	100	100	19-28
src/core/algorithms/assignment	83.45	77.04	85	83.21	
assignment-lifecycle.ts	85.71	77.77	83.33	85	14-15,54
greedy-assignment.engine.ts	83.05	76.92	85.71	82.9	143,276-304
src/core/algorithms/feedback	85.71	85.71	100	85.71	
feedback-updater.ts	85.71	85.71	100	85.71	6
src/core/algorithms/filters	82.14	83.33	81.81	84	
eligibility.filter.ts	82.14	83.33	81.81	84	23,46,57,70
src/core/algorithms/ranking	46.87	35.71	57.14	53.57	
top-k-ranker.ts	46.87	35.71	57.14	53.57	19,35,41,54-74
src/core/algorithms/scorers	97.95	89.65	100	97.91	
academic.scorer.ts	96.29	88.88	100	96	55
composite.scorer.ts	100	100	100	100	
fairness.scorer.ts	100	100	100	100	
preference.scorer.ts	96.55	81.81	100	96.55	97
schedule.scorer.ts	100	100	100	100	
src/core/algorithms/utils	100	96.15	100	100	
max-heap.ts	100	100	100	100	
vector-math.ts	100	87.5	100	100	2
src/core/constants	100	100	100	100	
defaults.ts	100	100	100	100	
index.ts	100	100	100	100	
src/core/engine	70	100	50	66.66	
index.ts	100	100	100	100	
matching-engine.ts	62.5	100	40	62.5	27,35-43
src/core/entities	93.05	77.08	100	92.18	
algorithm-weights.vo.ts	91.66	56.25	100	90.9	88,94
availability-slot.vo.ts	83.33	80	100	83.33	12,16
criterion-weights.vo.ts	95.83	90.9	100	95.45	71
index.ts	100	100	100	100	
match-score.vo.ts	100	100	100	100	
src/core/enums	100	100	100	100	
assignment-status.enum.ts	100	100	100	100	
delivery-mode.enum.ts	100	100	100	100	
exam-type.enum.ts	100	100	100	100	
format-preference.enum.ts	100	100	100	100	
index.ts	100	100	100	100	
learning-pace.enum.ts	100	100	100	100	
learning-style.enum.ts	100	100	100	100	
teaching-style.enum.ts	100	100	100	100	
src/core/evaluation	82.79	58.64	72.56	82.83	
baseline-comparison.ts	82.19	60.86	66.66	83.84	288-323,347-348

Figure 4.15: Core-layer test coverage output (pnpm test:cov, 28 September 2026)

At the final full pnpm test:cov run (28 September 2026), all 197 core-layer tests passed (including 23 in core-engine.spec.ts and 55 in core-units.spec.ts), the application-level health-check assertion in app.controller.spec.ts passed, and the full repository suite passed 334 of 355 tests across 21 suites, with the 21 skips all belonging to the one database-dependent courses integration suite. Table 4.9 transcribes the per-class coverage from that run.

4.9.2 Edge Case Verification

All twelve edge cases identified during design (Section 4.6) and the structural displacement case added with the repair pass were directly confirmed via named tests in core-engine.spec.ts and core-units.spec.ts:

Table 4.10: Edge Case Verification

Edge Case	Test Result	Verification
Zero eligible tutors after filter	PASS	Filtered into an explicit waitlist result with reason
Tutor at full capacity (assignedCount = capacity)	PASS	EligibilityFilter excludes the tutor; verified by named capacity tests
Tutor with capacity set to zero	PASS	Zero-capacity tutors are filtered into explicit waitlist results; FairnessScorer returns 0
Preference weights don't sum to 1	PASS	CriterionWeights.from() normalizes before scoring
Student with zero availability slots	PASS	Rejected as an incomplete profile before scoring
Availability request split across non-contiguous tutor slots	PASS	Split tutor slots are not treated as full coverage for one contiguous request
Tie in match score between tutors	PASS	Deterministic hash-based tie-break
New tutor with null average rating	PASS	Cold-start quality default applied without error
Tutor slot filled mid-batch	PASS	Capacity rechecked during the same batch
Incremental request at full capacity	PASS	Waitlisted rather than erroring
Assignment cancelled	PASS	Cancellation frees capacity and promotes a waitlisted student
Oversubscription (demand exceeds aggregate capacity)	PASS	Excess students are returned as unassignable, each with an explicit reason
Student left out while a seat is held by a student with an acceptable alternative	PASS	Bounded repair pass re-routes the seated student; new reachable placements are bounded by free capacity (Algorithm 4.1 step 4)

Beyond these named verifications, a randomized stress test (25 generated fixtures of 5 tutors and 20 students each) confirms that assigned counts remain within tutor capacity in every run, guarding the capacity invariant under arbitrary inputs rather than hand-picked cases.

4.9.3 Test Execution

The suite is run with `pnpm run test:cov` from the backend package. Tables 4.9 and 4.10 were transcribed from the final authoritative run of 28 September 2026, in which 334 of 355 tests passed across 21 suites (21 skipped in the database-dependent courses integration suite), including all 197 core-layer tests, the 49 evaluation-TUI tests, and the solver exactness suite of Section 4.10.6.

4.10 Results and Evaluation

4.10.1 Evaluation Harness

The datasets instantiate the domain profile of Section 3.1, the subjects, grade levels, examination types, availability patterns and tutor ranges of the Nigerian secondary-school setting. Results on that profile speak to the problem the requirements described.

Two harnesses are used. The scale harness (`core/evaluation/evaluation-harness.ts`) generates synthetic datasets of configurable size and ratio for single-population measurement without live production data (Section 1.6). The multi-population suite (`core/evaluation/baseline-statistics.ts`) runs the same fixtures over 30 independent populations (exploratory seeds 0-29), which supply every table in Sections 4.10.2 to 4.10.8 except the single-population wall-clock run of Table 4.13. Three scenario families were run: realistic-1to1, the moderate band (1.5:1, 2:1, 3:1) and a 10:1 stress market. Arms are compared with means, 95% confidence intervals and a paired sign test, and a difference is reported only when it separates at $p < 0.05$.

4.10.2 Match Quality and Fairness

Match quality and load fairness were measured using the average compatibility score, the percentage of students left unassigned, and Jain's fairness index (Jain et al., 1984) over tutor loads:

where i is tutor i 's assigned load and n is the number of tutors. A value of 1 indicates perfectly even load distribution.

Table 4.11: Match Quality and Fairness by Scenario (deployed engine; 30 independent populations per scenario; loadFactorWeight = 0.05)

Scenario	Ratio	Students	Tutors	Avg Compatibility Score	Unassigned %	Jain Fairness Index
realistic-seed	1:1	50	50	0.5901 +/- 0.0068	7.13%	0.5060 +/- 0.0139
moderate-1.5 to 1	1.5:1	150	100	0.6197 +/- 0.0039	1.93%	0.6420 +/- 0.0114
moderate-2to 1	2:1	150	75	0.5931 +/- 0.0037	5.38%	0.7302 +/- 0.0104
moderate-3to 1	3:1	150	50	0.5704 +/- 0.0060	17.51%	0.8521 +/- 0.0120
stress-sweep	10:1	1,000	100	0.7064 +/- 0.0026	75.00%	0.8333

Table 4.11 reports the five scenarios of the multi-population design: the 1:1 seed shape, the moderate band, and the 10:1 stress market at 1,000 students and 100 tutors. The 5,000-student tier (Table 4.13) and the supply-rich pair of Section 4.10.4 are measured separately, outside that design. Every figure is a mean over 30 populations with its 95% confidence interval.

Two effects should be read separately. Unassigned percentage rises with the student:tutor ratio, from 7.13% at 1:1 to 75.00% at 10:1, and is governed almost entirely by aggregate capacity rather than the assignment rule. Jain's index also rises with the ratio, from 0.5060 at 1:1 to 0.8333 at 10:1, for a different reason: with more students per tutor, more tutors carry similar, higher loads, and in the stress market capacity alone fixes the load vector. These percentages already include the repair pass, which seats a further 0.77 students per population at 1:1, 3.27 at 1.5:1, 7.03 at 2:1 and 9.43 at 3:1, and nothing at 10:1.

4.10.3 Effect of the Fairness Weight

The fairness weight's effect was isolated by re-running every scenario above with the load factor weight (δ) at 0 as well as at its designed value of 0.05:

Table 4.12: Effect of the Fairness Weight by Ratio (deployed engine; 30 independent populations per scenario; $\delta = 0.05$ against $\delta = 0$)

Ratio	Jain Index ($\delta=0.05$)	Jain Index ($\delta=0$)	Difference	Avg Score ($\delta=0.05$)	Avg Score ($\delta=0$)
1:1	0.5060	0.4989	+0.0071	0.5901	0.5947
1.5:1	0.6420	0.6355	+0.0065	0.6197	0.6278
2:1	0.7302	0.7133	+0.0168	0.5931	0.6108
3:1	0.8521	0.8233	+0.0288	0.5704	0.5983
10:1 (n=1,000)	0.8333	0.8333	+0.0000	0.7064	0.7064

Turning the weight on lifts Jain's index at every ratio measured: +0.0071 at 1:1, +0.0065 at 1.5:1, +0.0168 at 2:1, +0.0288 at 3:1, no change at 10:1 where capacity fixes the load vector. The score column looks like a cost of fairness but is not one: the $\delta = 0$ arm reports a 5.15% to 5.33% higher static total per student, and weight normalisation alone predicts it, since with $\delta = 0$ the remaining criteria are rescaled by $1/0.95$, inflating every static term by 5.26%. The match holds to within 0.1 percentage points in all five scenarios, so the difference is definitional; the $\delta = 0$ arm is included only to isolate the load-balancing effect and must not serve as a quality baseline.

Figure 4.10 plots the fairness column of Table 4.12, with 95% confidence intervals across the 30 populations.

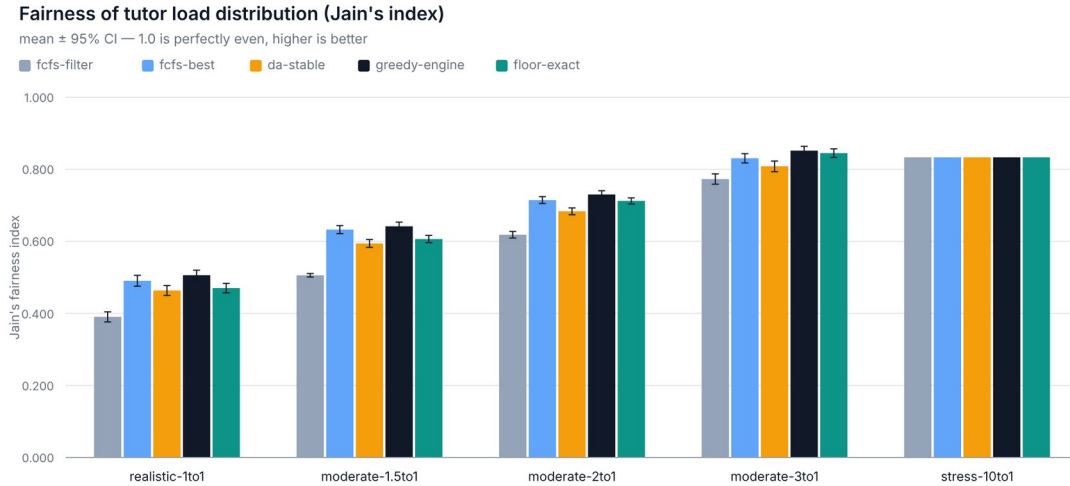


Figure 4.10: Fairness by ratio (Jain's index, engine with and without the fairness weight; 95% confidence intervals)

A focused unit test isolates the mechanism from this aggregate, capacity-driven effect. Given two otherwise identical tutors, one nearly full (2/3 capacity) and one empty (0/3), a single student's total match score favoured the empty tutor by 0.0359 when `loadFactorWeight` = 0.05, and the two tutors scored identically to six decimal places when `loadFactorWeight` = 0. That confirms the fairness term on a fixture small enough to check by hand, independently of the aggregate comparison above.

4.10.4 Performance Benchmarking

To validate the $O(n^2 \log n)$ theoretical bound from Section 4.5.3, wall-clock execution time was measured on the stress-sweep scenario at four scales, each run 5 times:

Table 4.13: Execution Time by Scale (stress-sweep, deployed engine, `loadFactorWeight` = 0.05; five runs per scale, 28 September 2026)

Students (S)	Tutors (T)	Pairs Scored	Elapsed Min (ms)	Elapsed Mean (ms)	Elapsed Max (ms)
50	5	14	1	0.8	1
200	20	396	5	5.3	6

1,000	100	6,769	57	65.7 (p50 61 / p95 83)	88
5,000	500	176,790	1,723	1,740.9 (p50 1,731 / p95 1,763)	1,766

Pairs scored rise from 396 at 200 students to 6,769 at 1,000 and 176,790 at 5,000, a 17-fold then 26-fold step across two five-fold scale increases, consistent with the scoring floor; mean elapsed time grows from 5.3 ms to 65.7 ms to 1,740.9 ms, matching the bound of Section 4.5.3. The 200-student count includes pairs the repair pass re-scores while searching; at 1,000 and 5,000 every seat is taken and the pass returns immediately. The 5,000-student run places the market's seat bound of 25% in 1.74 s, so the cost is scoring, not assignment. The exact floor-constrained solver is cheap on the same seat-bound market (111.7 ms at 1,000x100, 521.1 ms at 2,000x200) and prohibitive only once seats stop binding, as the next paragraph shows.

A second run measured the same chain on a supply-rich market, 1,000x600 and 2,000x1,200, where seats do not bind. The engine completes both in 0.62 s and 1.75 s with coverage 1.0000; the exact floor solve needs 2.68 s and 36.19 s, the unconstrained oracle 7.93 s and 63.76 s (capped at 2,000 augmentations, reported as partial). Two things stand out. The repair pass earns its cost where it is needed: nothing to do on the saturated 5,000x500 tier, but seventeen further students seated at 2,000x1,200. And the solver's cost follows the flow value, not the graph size: 1,000x100 places 250 students in 111.7 ms while 1,000x600 places 1,000 in 2.68 s, the same student count for roughly twenty-four times the work.

4.10.5 Comparison with Baseline Assignment Strategies

RQ6 asks how the engine compares with the methods tutoring platforms traditionally use. Three baselines run on the same fixtures, eligibility filter and composite scorer, so any difference reflects the assignment strategy alone. Filter-only FCFS assigns each student in arrival order to the first eligible tutor with spare capacity, mirroring deployed tutor-finding applications (Taveekarn et al., 2014); self-selection FCFS lets each student take their own highest-scoring eligible tutor, per-student optimal with no coordination across students.

The third, deferred acceptance (da-stable), is the student-proposing Gale-Shapley algorithm (Gale & Shapley, 1962; Roth, 2008) in its college-admissions form with tutor capacities: both sides rank by the same static composite score and the outcome is stable, with no pair preferring each other over their final matches. Stability is a different objective from total score, so it is a genuine competing alternative rather than a tuning variant.

Table 4.14: Baseline Comparison by Scenario (identical fixtures and scorer per scenario; 30 independent populations per arm; 28 September 2026)

Scenario	Avg (filter FCFS)	Avg (self- select)	Avg (deferred acceptance)	Avg (this engine)	Jain (filter FCFS)	Jain (self- select)	Jain (deferred acceptance)	Jain (this engine)
realistic- seed (1:1)	0.5226	0.5916	0.5942	0.5901	0.3905	0.4907	0.4636	0.5060
moderate- 1.5to1	0.5258	0.6193	0.6267	0.6197	0.5058	0.6327	0.5942	0.6420
moderate- 2to1	0.5256	0.5980	0.6102	0.5931	0.6185	0.7147	0.6835	0.7302
moderate- 3to1	0.5247	0.5744	0.5984	0.5704	0.7730	0.8308	0.8083	0.8521
stress- sweep (10:1)	0.5240	0.5822	0.7061	0.7064	0.8333	0.8333	0.8333	0.8333

Two metrics are needed here, and they disagree. On the load-independent total score per student the engine leads every baseline: 30 of 30 populations against filter-only FCFS at every ratio, and against both other baselines throughout the contested 1.5:1, 2:1 and 3:1 band. In the 1:1 seed scenario it wins 20 of 30 against self-selection ($p = 0.0987$) and 21 of 30 against deferred acceptance ($p = 0.0428$); at 10:1, 24 of 30 against deferred acceptance ($p = 0.0014$). The margin over self-selection is +0.0046 at 1:1, +0.0097 at 1.5:1, +0.0186 at 2:1, +0.0269 at 3:1 and +0.0310 at 10:1 per student. On coverage it also leads everywhere, 92.87% against 91.93% at 1:1 and

82.49% against 77.91% at 3:1, and leaves fewer students unassigned than any other arm in four of the five scenarios.

The second metric, average compatibility over placed matches, is deliberately not the headline here: it averages only over seated students, so it flatters whichever strategy leaves more unplaced. Read that way, self-selection is indistinguishable from the engine at 1:1 (0.5916 against 0.5901, $p = 0.8555$) and 1.5:1 (0.6193 against 0.6197, $p = 0.8555$), ahead at 2:1 (0.5980 against 0.5931, $p < 0.0001$) and marginally at 3:1 (0.5744 against 0.5704, $p = 0.2005$), while the engine is far ahead at 10:1 (0.7064 against 0.5822, $p < 0.0001$). The disagreement is the coverage difference itself, which is why the load-independent column is the one to quote. Against filter-only FCFS the engine leads on both metrics at every ratio: 9% to 35% higher average compatibility and a 15% to 37% higher total per student.

Figure 4.11 shows the two metrics separating: coverage by strategy, where the arms sit within two percentage points of one another at 1:1 and pull apart as contention rises.

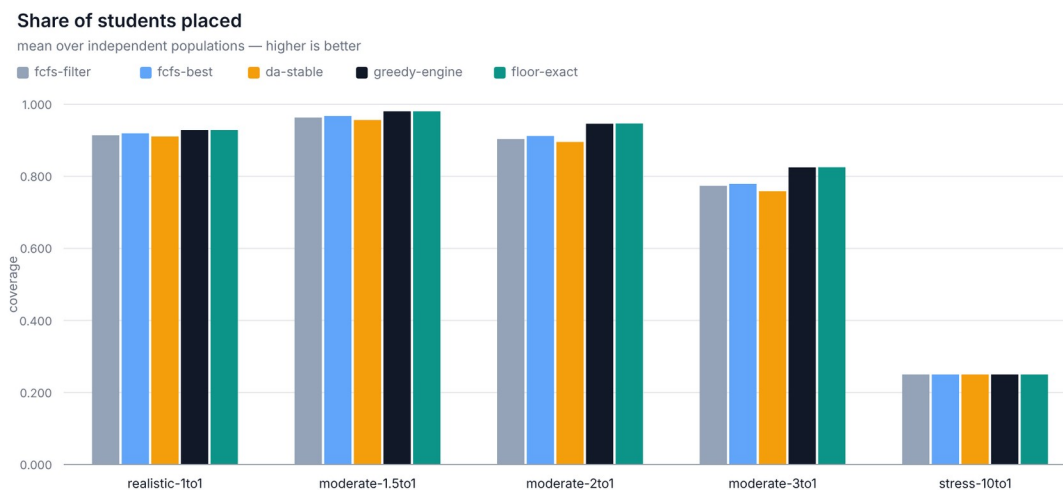


Figure 4.11: Coverage by strategy (share of students placed; 30 populations per scenario)

Figure 4.12 shows that placed-only mean, with overlapping intervals wherever the strategies are not distinguishable.

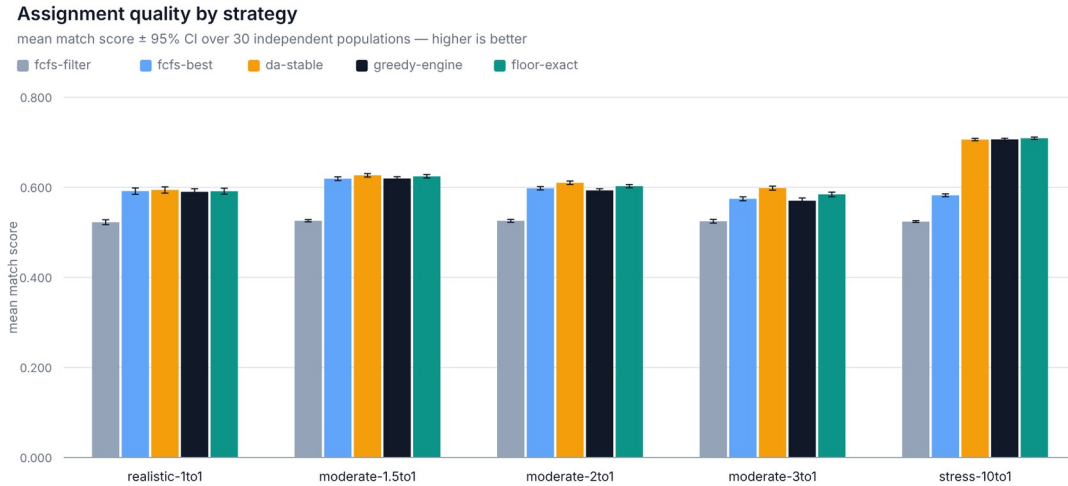


Figure 4.12: Placed-only average match score by strategy (30 populations per scenario; overlapping intervals are not distinguishable)

4.10.6 The Fairness Floor and the Worst-Placed Student

Neither metric says anything about the weakest match a student receives. The worst placed pair of Table 4.15, 0.373 to 0.470 across the five scenarios, is the number a fairness claim has to defend, and this section bounds it exactly. A fairness floor θ refuses any pair scoring below θ . Two quantities are then exact: the ceiling, the largest θ at which every student remains assignable using only pairs at or above it, found by binary search with an exact b-matching feasibility test; and the floor-constrained assignment, a min-cost max-flow solve that maximises placements then static total subject to every pair clearing θ . Both use the static composite score, since the δ fairness term depends on live load.

Table 4.15 gives the result over the 30 populations per scenario. The exact floor-constrained solve runs at the engine's own floor θ_0 , which is what makes the comparison like-for-like: the solve is given the engine's worst placed pair as its constraint, and may only do better from there.

Table 4.15: The fairness floor and the worst-placed student (deployed engine vs the exact floor-constrained solve; 30 independent populations per scenario; 28 September 2026)

Scenario	Engine floor θ_0	Exact ceiling ($\theta^* - \theta_0\theta_0$)	Worst placed pair: engine $\rightarrow$ floor-constrained solve
----------	----------------------------	--	--

realistic-seed (1:1)	0.402581	0.410117 (+0.007536)	0.402581 → 0.409201
moderate-1.5:1	0.382027	0.405856 (+0.023829)	0.382027 → 0.396063
moderate-2:1	0.373943	0.397045 (+0.023102)	0.373943 → 0.386643
moderate-3:1	0.372528	0.396635 (+0.024107)	0.372528 → 0.384806
stress-sweep (10:1)	0.469588	0.489518 (+0.019930)	0.469588 → 0.482562

The ceiling exceeds the engine's own floor by +0.0075 to +0.0241 in every scenario, 3.8 to 12 times the 0.002 negligibility band, so the tail has real headroom; it is a property of the assignment rule, not of the eligibility gates or capacity. Imposing the floor and re-solving exactly lifts the worst placed pair in every scenario and never lowers the total: 30 of 30 populations on the load-independent total ($p < 0.0001$) and on the worst placed pair ($p \leq 0.0039$). Coverage is unchanged or better, reported as “never worse” because it ties on most populations and wins on a handful.

The price of the floor is small enough to be the headline: at the ceiling the exact static total per student is slightly higher than the engine's own (+0.0007 to +0.0067), because the greedy total is not on the frontier, and the solve reaches 99.98% to 100.00% of the unconstrained optimum. Inside the frontier, raising the floor from θ_0 to the ceiling costs at most 0.000211 of static total per student with coverage held. At this level fairness is not a trade against quality.

The exact solve stays out of production. Its cost is set by the number of placements rather than the graph size: 2.68 s on the 1,000x600 market and 36.19 s on 2,000x1,200, against 1.74 s for the whole deployed engine at 5,000 (Section 4.10.4), and the ceiling search needs a further ~150 ms at the 150-student tier and ~27 s at 5,000. The floor is therefore an offline or administrative re-solve (Section 5.5); production keeps the greedy engine. Tail figures are quoted on the static basis only, since the solver cannot model the live δ load term.

The lift is not confined to the small fixtures. On the supply-rich market of Section 4.10.4 the exact solve raises the worst placed pair from 0.3879 to 0.4493 at 1,000×600 and from 0.4197 to 0.4857 at 2,000×1,200, with coverage equal to the engine's. That is a larger lift than the 150-student tier gives, obtained while every student is still placed.

4.10.7 Stability of the Matching

Every result so far scores a matching; none asks whether the two sides would accept it. A pair (student, tutor) blocks a matching when the student is unplaced or prefers that tutor, the tutor has a spare seat or scores its weakest holder below the pair's static score, and the pair clears eligibility. A matching with no blocking pair is stable in Gale and Shapley's (1962) sense, which is why deferred acceptance must report zero blockers.

That anchor keeps the count honest, and it holds: deferred acceptance reports exactly 0.00 blocking pairs in all five scenarios over 30 populations. Against it, the deployed engine leaves 4.93 blockers per population at 1:1 and 25.43, 33.53 and 42.03 at the three contested ratios (Table 4.16). Stability is the one measured property on which the engine, rather than its baselines, is the weakest arm.

Table 4.16: Blocking pairs per population by strategy (mean over 30 populations, exploratory seeds 0–29; 0 = a stable matching)

Scenario	Filter FCFS	Self- selection	Deferred acceptance	This engine	Stability pass
realistic-seed (1:1)	66.43	4.90	0.00	4.93	1.63
moderate-1.5:1	425.57	38.83	0.00	25.43	11.13
moderate-2:1	296.83	45.67	0.00	33.53	20.20
moderate-3:1	193.87	59.57	0.00	42.03	28.17
stress-10:1	3931.10	2280.10	0.00	3.33	3.00

Two structural findings fall out of the table. The engine sits in the middle of the field rather than at its head, and the mechanism that lifts its coverage in Section 4.10.5 is the same one that costs it stability: displacing a seated student is how a blocking pair is created. Arrival-order assignment is

the other extreme, at 194 to 3,931 blockers per population, so a deployment that treats stability as a hard requirement can rule out filter-only FCFS on this evidence alone.

Figure 4.13 plots the same counts per student, with deferred acceptance flat on the zero line.

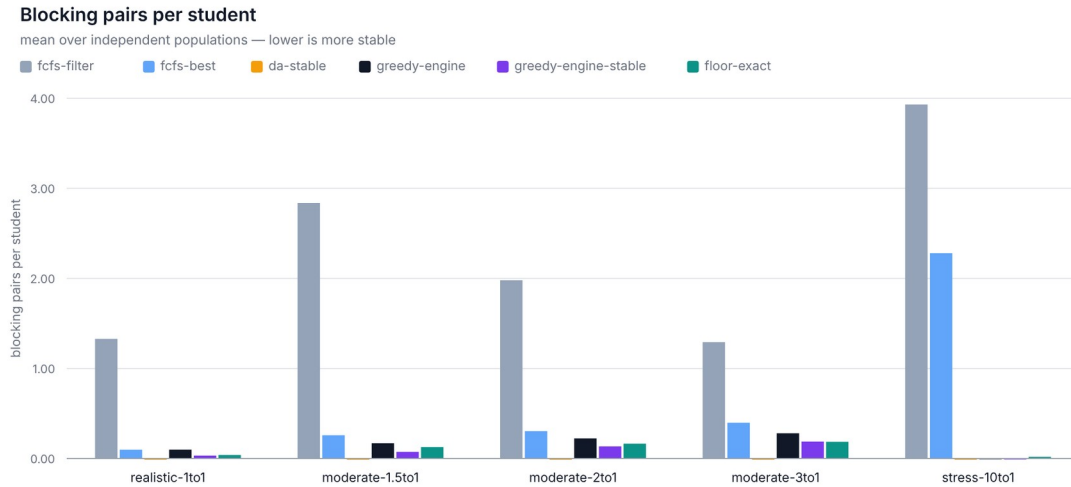


Figure 4.13: Blocking pairs per student by strategy (0 = a stable matching; 30 populations per scenario)

A bounded, deterministic stability pass was built to measure what recovering some of that costs. It runs after repair, resolving blockers one at a time: a free-seat move when the tutor has room, otherwise a swap with the weakest displaceable holder; it accepts a resolution only when the blocking count strictly falls, and stops after 64 accepted moves. It is opt-in (`assignBatch(..., { stability: true })`) and off by default. Over 30 populations it removes 3.30 blockers at 1:1 and 14.30, 13.33 and 13.87 at the contested ratios (30 of 30, $p < 0.0001$), loses no placement, and raises the load-independent total on average, though it lowers it in three populations at 1:1 and one at each contested ratio.

Table 4.17: Cost of the stability pass (stability pass minus deployed engine; 30 populations; 28 September 2026)

Scenario	Δ Blocking pairs	Δ Coverage	Δ Total per student	Δ Jain fairness	Δ Worst placed pair
realistic-seed	-3.30	0.000000	+0.000534	-0.032094	-0.006660

(1:1)					
moderate-1.5:1	-14.30	0.000000	+0.000999	-0.034923	-0.005159
moderate-2:1	-13.33	0.000000	+0.001716	-0.018344	-0.006621
moderate-3:1	-13.87	0.000000	+0.002219	-0.007133	-0.009678
stress-10:1	-0.33	0.000000	-0.000006	+0.000000	+0.000000

The pre-registered expectation was that the pass would buy stability out of static total. It does not; the tradeoff is three-way. Total rises very slightly (+0.0005 to +0.0022 per student, in 27 to 29 of 30 populations), coverage is identical to six decimals, but load fairness falls (Jain -0.007 to -0.035) and so does the worst-placed student (-0.005 to -0.010). The swaps that remove blockers push seats onto emptier tutors, the opposite of what the fairness weight steers towards, so stability and the fairness tail are in direct conflict. The pass costs 19.25 ms at p95 on the 150x100 tier, inside its 250 ms budget.

All of it reproduces on the held-out seeds: 2.73, 12.53, 14.73 and 13.47 blockers removed per population with 30 wins and no losses ($p < 0.0001$), coverage unchanged, every fairness and tail sign intact (Section 4.10.8). Because the pass pays for stability out of the tail that Section 4.10.6 bought back, it stays out of the request path, available as an explicit opt-in.

4.10.8 Statistical Analysis of the Evaluation

Every scenario is evaluated over 30 independent populations, because a single fixture set cannot separate a real effect from fixture noise. Three tools turn samples into claims: the mean with its 95% confidence interval (Student-t, 29 degrees of freedom); the exact two-sided paired sign test against the deployed engine, so a mean that hides per-population losses cannot pass; and Cohen's d_z for effect size. Differences below 0.002 on a score in $[0, 1]$ were defined in advance as negligible, and every comparison is judged against that band as well as $p < 0.05$. Jain's index and the Gini coefficient moved in opposite directions in every comparison, so only Jain is carried into the tables.

Table 4.18 collects the inferential results for the chapter's headline metric, the load-independent total score per student, together with the paired tallies behind the claims of Sections 4.10.5 and

4.10.6. Each entry is a wins/losses/ties count for the named strategy against the deployed engine, followed by the two-sided p-value.

Table 4.18: Significance of the headline comparisons on the load-independent total per student (30 populations per scenario; W/L/T is the paired sign test against the deployed engine; 28 September 2026)

Stability pass vs engine	27/3/0 <0.0001	29/1/0 <0.0001	28/2/0 <0.0001	29/1/0 <0.0001	3/4/23 1.0000
Floor-constrained solve vs engine	30/0/0 <0.0001	30/0/0 <0.0001	30/0/0 <0.0001	30/0/0 <0.0001	30/0/0 <0.0001

Three readings stand out. Filter-only FCFS loses all 30 populations in every scenario and self-selection loses all 30 from 1.5:1 upward, which is the quantitative case for scoring at all. The 1:1 seed scenario is genuinely undecided between engine and self-selection (10 wins, 20 losses, $p = 0.0987$), underpowered rather than settled; the engine's edge begins at 2:1, where contention makes score-aware ordering pay. The two mechanisms added on top of plain greedy behave differently by regime: the floor-constrained solve wins the load-independent total in 30 of 30 populations at every ratio, and the repair pass never loses, though at 1:1 it improves the total in only half the populations.

Figure 4.14 collects every baseline-against-engine difference behind Table 4.18; bars left of zero are populations the engine won, and 15 of the comparisons are significant at $p < 0.05$.

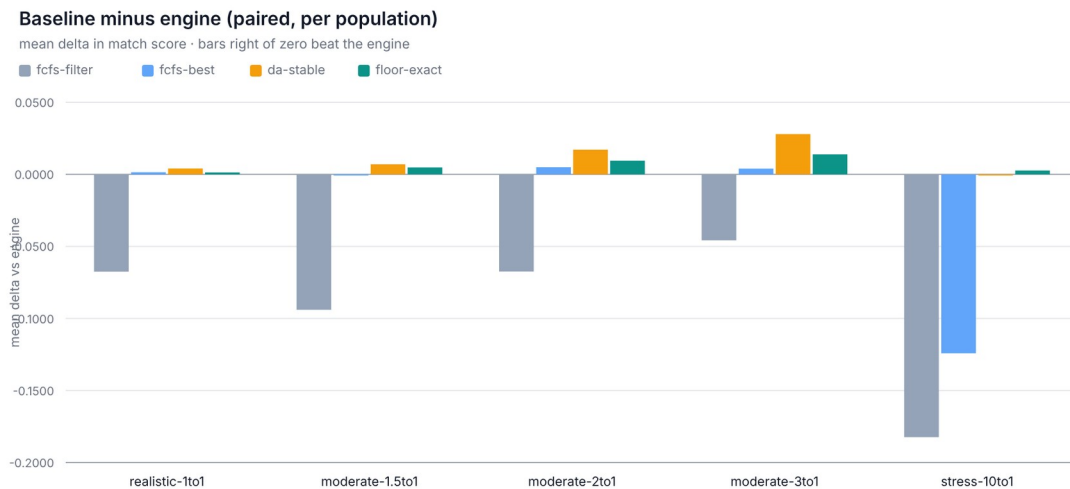


Figure 4.14: Each baseline against the engine on the load-independent total per student (mean per-population difference; 30 populations per scenario)

The same machinery applied to the worst-placed student gives the chapter's central fairness result its statistical form. The floor-constrained solve beats the deployed engine on the worst placed static score in 9, 19, 13 and 18 of 30 populations at 1:1, 1.5:1, 2:1 and 3:1, and in 19 of 30 at 10:1, with no losses in any scenario ($p \leq 0.0039$), while its total never falls. The tail can be lifted without paying for it.

The last piece of evidence tests the analysis rather than the algorithm. The hypotheses, guardrails and acceptance rules of every stage were chosen against populations drawn from seeds 0–29, and those seeds had a hand in the design, so they cannot also confirm it. Seeds 1000–1029 were reserved before the runs and spent once on a single sweep whose result was pre-registered as publishable whatever it showed. Table 4.19 gives the outcome.

Table 4.19: Held-out confirmation (seeds 1000–1029): exploratory value → held-out value for each registered direction claim

Registered claim	1:1	1.5:1	2:1	3:1
Engine – self-selection, coverage	+0.0093 → +0.0073	+0.0131 → +0.0169	+0.0342 → +0.0278	+0.0458 → +0.0411
Engine – filter FCFS, coverage	+0.0147 → +0.0127	+0.0173 → +0.0204	+0.0427 → +0.0331	+0.0513 → +0.0484
Repair gain in coverage	+0.0153 → +0.0153	+0.0218 → +0.0256	+0.0469 → +0.0467	+0.0629 → +0.0587
Floor solve – engine, worst static	+0.0066 → +0.0031	+0.0140 → +0.0090	+0.0127 → +0.0169	+0.0123 → +0.0128
Floor solve – engine, total (must be ≥ 0)	+0.0013 → +0.0012	+0.0040 → +0.0042	+0.0061 → +0.0056	+0.0067 → +0.0071
Engine – stability pass, blocking pairs	+3.30 → +2.73	+14.30 → +12.53	+13.33 → +14.73	+13.87 → +13.47
Engine static total vs	0.9975 →	0.9931 →	0.9886 →	0.9852 →

oracle (must be < 1)	0.9974	0.9926	0.9894	0.9841
----------------------	--------	--------	--------	--------

All six registered direction claims hold on the held-out seeds, and twenty-three of the twenty-four effect sizes land within $\pm 50\%$ of their exploratory estimate. The exception, the 1:1 tail lift, came in 53% smaller ($+0.0066 \rightarrow +0.0031$); the registered rule reports that as direction reproduced with a smaller effect size, and the lift is quoted from here on as the optimistic end of that range. Two invariants also hold: deferred acceptance reports zero blocking pairs on all 150 held-out populations, and the unplaced-cause buckets still sum exactly to the unplaced count. Nothing inverted, and no claim moved by more than 53% in either direction.

The limits matter as much as the results. A p-value between 0.05 and 0.2 is reported as underpowered, not as evidence of no effect; seeds 1000–1029 are spent, so further confirmation needs a fresh registered range; and neither the floor frontier nor the scale benchmark was re-run held-out, so neither is described as confirmed. Both are exact per population or measured at sizes a confirmation sample could not change.

4.10.9 Explainability

The system’s per-match score breakdown is one artifact that needs no benchmark run: every assignment returns a JSON structure decomposing the total score into its four components and their sub-components:

Sample Score Breakdown (structure)

```
{
  "total": 0.0,
  "breakdown": {
    "academic": 0.0,
    "preference": 0.0,
    "schedule": 0.0,
    "fairness": 0.0
  },
  "subBreakdown": {
    "subjectDepth": 0.0,
    "level": 0.0,
    "experience": 0.0,
    "style": 0.0,
    "budget": 0.0,
    "region": 0.0
  }
}
```

The client renders this breakdown as a percentage alongside a per-criterion explanation, making the algorithm’s decision inspectable rather than a black box, which directly addresses RQ7.

4.11 Discussion

The evaluation rests on three separate, evidence-graded claims, and they should not be flattened into one.

Proven, not measured: the 1/2-approximation guarantee for greedy weighted bipartite matching (Preis, 1999; Korte & Vygen, 2018) is a mathematical property and holds whatever the input data. It applies to the static, no-repair, untruncated variant, because the deployed engine departs from it in four documented ways (fairness in the priority, top-k truncation, hash tie-breaks, and the repair pass). The bound is therefore quoted as the guarantee for the underlying matching step, not as a claim about the shipped system.

Measured and strong: against a true optimal baseline computed by an exact min-cost max-flow solver at small scales, the deployed engine matched 97.6-100% of the optimal total score (Section 4.5.5). On 30 independent populations per scenario it keeps 98.5% to 99.7% of the oracle static total and beats every baseline on the load-independent total: 30 of 30 against filter-only FCFS at every ratio, 30 of 30 against both other baselines across the contested 1.5:1–3:1 band, and 24 of 30 against deferred acceptance in the supply-bound 10:1 market. It does so at 1.74 s for 5,000 students against 63.8 s for the exact solver at 2,000, which is the empirical case for the engine rather than the solver in production.

Measured and regime-dependent. The fairness weight shifts load balance at every ratio measured: Jain’s index rises by +0.0071 at 1:1, +0.0065 at 1.5:1, +0.0168 at 2:1 and +0.0288 at 3:1, and is unchanged at 10:1, where tutor capacity rather than the assignment order fixes the load vector (Table 4.12). The score column in that comparison is not evidence of a cost: the $\delta = 0$ arm’s apparent 5.26% advantage is reproduced to within 0.1 percentage points by the weight renormalisation the arm itself performs, so it is a bookkeeping artifact. What the weight genuinely does is bounded, measurable and one-directional: it moves load. Its cost cannot be isolated by this ablation because the ablation changes the weighting before it changes anything else.

Static sorting uses stale information, because a tutor's fairness score changes as students are assigned during a batch; lazy recomputation fixes that at the same asymptotic cost as one sort (Section 4.5.3). The repair pass fixes the heap's other flaw, never revisiting a decision, which can leave a student out while a seat is held by someone with an acceptable alternative. Between them the two mechanisms place the engine ahead of every baseline on coverage and load-independent total, within 2.4% of the exact optimum at the largest instance tested.

CHAPTER FIVE

SUMMARY, CONCLUSION AND RECOMMENDATIONS

5.1 Summary of Findings

This project set out to solve how students get matched with suitable tutors in digital learning environments. Existing systems lean on limited filtering criteria and do not combine weighted compatibility, scheduling feasibility, fairness and explainability in one framework.

To close that gap a multi-criteria matchmaking system was designed, implemented and evaluated, resting on four sub-scorers (academic, preference, schedule, fairness), a priority-queue greedy assignment with lazy fairness recompute and a bounded repair pass, and an EMA feedback update.

It is a full-stack application: NestJS 11 on the backend, the core algorithm isolated in a framework-independent domain layer, and PostgreSQL with Drizzle ORM for persistence. The final run of 28 September 2026 executed 355 tests across 21 suites: 334 passed, 21 skipped, none failed, including all 197 core algorithm tests in nine spec files, with all 12 edge cases confirmed by named tests (Section 4.9.2).

Time complexity is , confirmed empirically by quadratic growth in pairs scored (Sections 4.5.3, 4.10.4). The classical $1/2$ -approximation guarantee holds for the static no-repair variant (Preis, 1999; Korte & Vygen, 2018); the deployed engine departs from it in four documented ways and is judged by measurement, reaching 97.6-100% of the true optimum at small scales and retaining 98.5-99.7% of the oracle static total over 30 populations per scenario while beating every baseline on coverage and load-independent score.

The harness (Section 4.10) recorded match quality, Jain's fairness index, coverage and execution time across the three scenario families, with every comparative figure a mean over 30 populations, a 95% confidence interval and a paired sign test. The fairness weight improves load distribution at every measured ratio (Jain's index $+0.0071$ to $+0.0288$, Table 4.12) and is inert at 10:1, the accompanying score difference being a normalisation artifact (Section 4.10.3). The worst-placed student was bounded exactly: the largest floor at which every student remains assignable sits

0.0075 to 0.0241 above the engine's own worst placed pair, and the exact assignment under that floor lifts the tail at no measured cost (Section 4.10.6).

Stability was the one axis where the engine is the weakest arm: deferred acceptance reports zero blocking pairs by construction, the engine leaves 4.93 to 42.03 per population, and the repair pass that closes the coverage gap creates most of them. The opt-in stability pass removes 3.30 to 14.30 with no loss of placement, but pays out of load fairness and the worst-placed student, so it is not deployed by default.

5.2 Conclusions

Student-tutor matching is better framed as a multi-criteria decision problem than as a filtering task. What the implemented system demonstrates is that four sub-scorers, priority-queue greedy assignment and load balancing yield matches that are fast, close to optimal, explainable and fair under realistic capacity conditions.

Each research question was answered as follows:

- RQ1: A multi-criteria weighted model with four sub-scorers ($M(s,t) = \alpha A + \beta P + \gamma S + \delta F$) was designed and implemented, with all sub-scores bounded to $[0,1]$ and confirmed by unit tests covering 96.3-100% of each scorer's statements.
- RQ2: Student preferences are incorporated dynamically through `CriterionWeights`, which auto-normalise to sum to 1 and map to the four-bucket `AlgorithmWeights`.
- RQ3: the `ScheduleScorer` applies a hard availability check before any scoring happens, so only feasible pairs are evaluated. A student with no availability at all is rejected as an incomplete profile.

RQ4: Fairness and load balancing use the `FairnessScorer` $F(t) = \min(1, (1 - \text{LoadRatio}(t))^{1.15} + b(t))$ with a cold-start boost for unused tutors. It improves Jain's fairness index at every ratio the multi-population design covers, by +0.0071 at 1:1, +0.0065 at 1.5:1, +0.0168 at 2:1 and +0.0288 at 3:1, with no measurable change at 10:1 once tutor capacity binds (Section 4.10.3). The apparent score cost of enabling it is an artifact of the weight normalisation rather than a measured trade-off.

- RQ5: the EMA-based FeedbackUpdater, with $\lambda = 0.3$, folds historical performance into ratings, creating a closed-loop improvement system. New tutors default to `COLD_START_QUALITY = 0.5`.
- RQ6: the algorithm scales as , confirmed empirically by pairs scored growing with roughly the square of the system size, and it completes the 5,000-student tier in 1.74 s. The classical 1/2-approximation guarantee holds for the static no-repair variant; the deployed engine is judged by measurement and reached 97.6-100% of true optimal score against an exact baseline at small scales. On identical fixtures it delivered 9-35% higher average compatibility than filter-only FCFS at every ratio, and on the load-independent total score it won all 30 populations against filter-only FCFS at every ratio, all 30 against both other baselines across the contested 1.5:1 to 3:1 band, and 24 of 30 against deferred acceptance in the supply-bound stress market, where the two arms finished within 0.0001 of each other (Section 4.10.5).
- RQ7: Explainability is provided through per-criteria score breakdowns at both the four-bucket level and the sub-criteria level, returned in every assignment result.

Of the six specific objectives in Section 1.3, the first five are addressed directly by the design, analysis and evaluation above. Objective 6, the full-stack web application, was delivered as the NestJS, Next.js and PostgreSQL system of Sections 4.2–4.4 and 4.8 but, consistent with Sections 1.5 and 4.1.1, the client was excluded from analytical evaluation, which concentrates on the algorithmic core.

Statistically, the study rested on inference rather than point estimates: means over 30 independent populations, 95% confidence intervals, exact paired sign tests, differences below 0.002 reported as negligible, and a held-out seed range (1000–1029) reserved to confirm or refute. All six registered direction claims held, with one effect size (the 1:1 fairness-tail lift) materially smaller than its exploratory estimate and reported as such.

5.3 Recommendations

A controlled deployment on real tutor and student data from a tutorial centre or school platform would validate the simulated results against actual behaviour, but no such data was used here: a live

trial needs its own ethical clearance and fresh participant consent, and the no-real-world-data statement in Section 3.1 still describes the work reported.

- The default weights should be checked with domain experts and tuned in light of observed usage and A/B testing.

The load-factor weight (δ) improves load distribution at every ratio measured and is inert only where capacity already fixes the load vector, so it should stay on; the score difference that accompanies it is a weight-renormalisation artifact rather than a cost of fairness (Section 4.10.3).

- The administrator dashboard should also track tutor load, failed matches, and match outcomes over time.
- Because the engine is measurably not stable, a deployment that treats stability as a first-class requirement should turn the opt-in stability pass on and accept its measured price in load fairness and the worst placed pair, rather than expect the default engine to be stable; the cost is tabulated per scenario in Section 4.10.7.

5.4 Contribution to Knowledge

The study makes the following contributions to knowledge:

A composite scoring architecture with four independently testable sub-scorers and a priority-queue greedy assignment engine with lazy fairness recompute and a bounded repair pass, achieving time complexity, with the classical 1/2-approximation guarantee rescoped to the static no-repair variant the deployed engine extends.

Empirical validation against an exact min-cost max-flow baseline, showing 97.6-100% of optimal score in practice and 98.5-99.7% of the oracle's static total over 30 populations, while beating the filter-based, self-selection and deferred-acceptance strategies deployed platforms use (Sections 4.10.5 and 4.5.5).

- An exact bound on the fairness tail rather than an average: the largest floor at which every student remains assignable was computed exactly, showing that the engine's worst placed pair sits 0.0075 to 0.0241 below it, and that a floor-constrained exact assignment lifts the

tail in 30 of 30 populations per scenario at a measured price of at most 0.000211 of static total per student (Section 4.10.6).

An evaluation methodology that separates a real effect from a fixture artifact: 95% confidence intervals and paired sign tests over 30 independent populations, the load-independent total distinguished from the placed-only average that flatters low-coverage strategies, and the apparent quality cost of the fairness weight traced to the weight renormalisation it performs (Sections 4.10.2, 4.10.3 and 4.10.5).

- An open-source implementation of the full-stack system whose core algorithm sits in a framework-independent domain layer, built on clean architecture principles with dependency inversion at the repository boundary.
- A measured account of stability as a third axis of matching quality, including the first direct comparison of blocking-pair counts between a greedy assignment engine, the two first-come-first-served strategies platforms deploy, and a stable-matching baseline, together with the bounded pass that trades load fairness for stability (Section 4.10.7).

5.5 Future Work

A later version could expose the exact min-cost max-flow solver that Section 4.10.6 already benchmarks as an administrative re-solve endpoint, and could re-run the optimality gap and the fairness floor on real platform data rather than the generated corpora.

- Future work should add transactional integrity guarantees at the persistence layer, so concurrent match requests cannot cause phantom-write conflicts during peak load.
- The platform should also watch for bias, handle cancellations, and moderate user ratings more strictly so the feedback system cannot be gamed.
- Longitudinal studies could measure what the matchmaking system does to actual student learning outcomes over an academic term.
- The algorithm could take on further criteria, such as personality compatibility, learning-pace matching, and real-time availability updates.

- To scale past 10,000 users, database-level pre-filtering (pushing hard constraint checks into SQL) and parallel processing strategies should be introduced.

Stability and fairness could be attacked together rather than in sequence: the measured conflict between them suggests a formulation that places a floor on the worst placed pair as a constraint inside the same solve that eliminates blocking pairs, which the present pipeline keeps as two passes.

REFERENCES

- Adomavicius, G., & Tuzhilin, A. (2005). Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions. *IEEE Transactions on Knowledge and Data Engineering*, 17(6), 734–749. <https://doi.org/10.1109/TKDE.2005.99>
- Burke, R. (2002). Hybrid recommender systems: Survey and experiments. *User Modeling and User-Adapted Interaction*, 12(4), 331–370. <https://doi.org/10.1023/A:1021240730564>
- Coppolillo, E., Mungari, S., Ritacco, E., Fabbri, F., Minici, M., Bonchi, F., & Manco, G. (2024). Algorithmic drift: A simulation framework to study the effects of recommender systems on user preferences. *arXiv preprint arXiv:2409.16478*. <https://arxiv.org/abs/2409.16478>
- da Silva, F. L., Slodkowski, B. K., da Silva, K. K. A., & Cazella, S. C. (2023). A systematic literature review on educational recommender systems for teaching and learning. *Education and Information Technologies*, 28(3), 3289–3328. <https://doi.org/10.1007/s10639-022-11341-9>
- Deldjoo, Y., Jannach, D., Bellogín, A., Difonzo, A., & Zanzonelli, D. (2024). Fairness in recommender systems: Research landscape and future directions. *User Modeling and User-Adapted Interaction*, 34(1), 59–108. <https://doi.org/10.1007/s11257-023-09364-z>
- Deschenes, M. (2020). Recommender systems to support learners' agency in a learning context: A systematic review. *International Journal of Educational Technology in Higher Education*, 17, Article 50. <https://doi.org/10.1186/s41239-020-00219-w>
- Gale, D., & Shapley, L. S. (1962). College admissions and the stability of marriage. *The American Mathematical Monthly*, 69(1), 9–15. <https://doi.org/10.1080/00029890.1962.11989827>
- Grand View Research. (2025). Online tutoring services market size & share report, 2024–2030. <https://www.grandviewresearch.com/industry-analysis/online-tutoring-services-market>
- Hevner, A. R., March, S. T., Park, J., & Ram, S. (2004). Design science in information systems research. *MIS Quarterly*, 28(1), 75–105. <https://doi.org/10.2307/25148625>
- Hien, N. T. T., Quynh, P. H., & Minh, V. Q. (2025). A comparative analysis of multi-criteria decision-making methods. *Engineering, Technology and Applied Science Research*, 15(5), 26369–26375. <https://doi.org/10.48084/etasr.12782>

- Jain, R., Chiu, D., & Hawe, W. (1984). A quantitative measure of fairness and discrimination for resource allocation in shared computer systems (DEC Research Report TR-301). Digital Equipment Corporation. <https://arxiv.org/abs/cs/9809099>
- Karaci, A. (2019). Intelligent tutoring system model based on fuzzy logic and constraint-based student model. *Neural Computing and Applications*, 31(8), 3619–3628. <https://doi.org/10.1007/s00521-017-3311-2>
- Kester, L., van Rosmalen, P., Sloep, P., Brouns, F., Kone, M., & Koper, R. (2007). Matchmaking in learning networks: Bringing learners together for knowledge sharing. *Interactive Learning Environments*, 15(2), 117–126. <https://doi.org/10.1080/10494820701332663>
- Konstan, J. A., & Riedl, J. (2012). Recommender systems: From algorithms to user experience. *User Modeling and User-Adapted Interaction*, 22(1–2), 101–123. <https://doi.org/10.1007/s11257-011-9112-x>
- Korte, B., & Vygen, J. (2018). *Combinatorial Optimization: Theory and Algorithms* (6th ed.). Springer. <https://doi.org/10.1007/978-3-662-56039-6>
- Lechuga, C. G., & Doroudi, S. (2023). Three algorithms for grouping students: A bridge between personalized tutoring system data and classroom pedagogy. *International Journal of Artificial Intelligence in Education*, 33(4), 843–884. <https://doi.org/10.1007/s40593-022-00309-y>
- Leung, J. Y. T. (2004). *Handbook of Scheduling: Algorithms, Models, and Performance Analysis*. Chapman and Hall/CRC. <https://doi.org/10.1201/9780203489802>
- Lotfian, E., & Kabgani, A. (2026). HiMARS: Hybrid multi-objective algorithms for recommender systems. arXiv preprint arXiv:2604.07572. <https://arxiv.org/abs/2604.07572>
- Nuseibeh, B., & Easterbrook, S. (2000). Requirements engineering: A roadmap. In *Proceedings of the Conference on the Future of Software Engineering* (pp. 35–46). ACM. <https://doi.org/10.1145/336512.336523>
- Peppers, K., Tuunanen, T., Rothenberger, M. A., & Chatterjee, S. (2007). A design science research methodology for information systems research. *Journal of Management Information Systems*, 24(3), 45–77. <https://doi.org/10.2753/MIS0742-1222240302>

- Pinedo, M. L. (2022). *Scheduling: Theory, Algorithms, and Systems* (6th ed.). Springer. <https://doi.org/10.1007/978-3-031-05921-6>
- Pitoura, E., Stefanidis, K., & Koutrika, G. (2022). Fairness in rankings and recommendations: An overview. *The VLDB Journal*, 31(3), 431–458. <https://doi.org/10.1007/s00778-021-00697-y>
- Preis, R. (1999). Linear time $1/2$ -approximation algorithm for maximum weighted matching in general graphs. In C. Meinel & S. Tison (Eds.), *STACS 99 (Lecture Notes in Computer Science, Vol. 1563, pp. 259–269)*. Springer. https://doi.org/10.1007/3-540-49116-3_24
- Ramesh, V. (2020). Developing an accurate method and algorithm to identify and match tutors with their respective students, according to intellectual level and teaching style. 2020 Intern Reports, 20. Illinois Mathematics and Science Academy. https://digitalcommons.imsa.edu/intern_reports_2020/20/
- Ricci, F., Rokach, L., & Shapira, B. (Eds.). (2022). *Recommender Systems Handbook* (3rd ed.). Springer. <https://doi.org/10.1007/978-1-0716-2197-4>
- Roth, A. E. (2008). Deferred acceptance algorithms: History, theory, practice, and open questions. *International Journal of Game Theory*, 36(3–4), 537–569. <https://doi.org/10.1007/s00182-008-0117-6>
- Sommerville, I. (2016). *Software Engineering* (10th ed.). Pearson. <https://www.pearson.com/en-us/subject-catalog/p/software-engineering/P200000003258/9780137503148>
- Spivakovsky, A., Poltoratskyi, M., Lemeshchuk, O., Denysenko, V., Karpov, I., & Revenko, Y. (2025). Algorithms for using online tutoring as a tool for personalization of learning. In *ICT in Education, Research, and Industrial Applications (CCIS 2359, pp. 178–193)*. Springer. <https://doi.org/10.1007/978-3-031-81372-6>
- Tadelis, S. (2016). Reputation and feedback systems in online platform markets. *Annual Review of Economics*, 8(1), 321–340. <https://doi.org/10.1146/annurev-economics-080315-015325>
- Tarus, J. K., Niu, Z., & Mustafa, G. (2018). Knowledge-based recommendation: A review of ontology-based recommender systems for e-learning. *Artificial Intelligence Review*, 50(1), 21–48. <https://doi.org/10.1007/s10462-017-9539-5>

- Taveekarn, W., Latthitham, R., Kittichareonjit, N., & Visoottiviseth, V. (2014). FindMyTutor: An Android application for matching students and private tutors. In Proceedings of the 2014 Third ICT International Student Project Conference (ICT-ISPC) (pp. 5–8). IEEE. <https://murex.mahidol.ac.th/en/publications/findmytutor-an-android-application-for-matching-students-and-priv/>
- Vanguard. (2024, August 12). WAEC releases 2024 WASSCE results. Vanguard Nigeria. <https://www.vanguardngr.com/2024/08/breaking-waec-releases-2024-wassce-results/>
- Wang, Z., Nabavi, S. R., & Rangaiah, G. P. (2023). Selected multi-criteria decision-making methods and their applications to product and system design. In A. J. Kulkarni (Ed.), Optimization Methods for Product and System Design (pp. 107–138). Springer. https://doi.org/10.1007/978-981-99-1521-7_7
- Zhang, Y., & Chen, X. (2020). Explainable recommendation: A survey and new perspectives. Foundations and Trends in Information Retrieval, 14(1), 1–101. <https://doi.org/10.1561/15000000066>

APPENDIX

Appendix A: Source Code

The full source code for the backend, the core algorithm layer, and the client application exceeds the length appropriate for inclusion in this document and is kept in the project's version-controlled repository rather than reproduced here in full. The repository contains:

- `backend/src/core/`: the framework-independent domain layer described in Section 4.2.2 (AcademicScorer, PreferenceScorer, ScheduleScorer, FairnessScorer, CompositeScorer, GreedyAssignmentEngine, MaxHeap, FeedbackUpdater, EligibilityFilter).
- `backend/src/modules/`: the NestJS application layer (controllers, services, guards, DTOs) that exposes the domain layer over HTTP.
- `backend/drizzle/`, the database schema and migration files corresponding to Section 4.3.1.
- `core/evaluation/`: the evaluation harness described in Section 4.10.
- `backend/test/`: the Jest test suite described in Section 4.9.

<https://github.com/sammythadev/student-tutor-platform>

The four representative excerpts in Section 4.7.2 (CompositeScorer, GreedyAssignmentEngine, FairnessScorer, FeedbackUpdater) illustrate the core logic referenced throughout Chapter Four and are reproduced there rather than here, to keep them next to their explanation.

Appendix B: Corrections Log (Seminar Proposal to Implemented System)

This section records the specific changes and improvements made between the original seminar proposal (February 2026) and the final implemented system. The Algorithm.md specification and its corrections log (section 10) guided these changes, systematically addressing issues found during implementation.

Table B.1: Changes between the Seminar Proposal and the Implemented System

Area	Seminar Proposal	Implemented System	Rationale
------	------------------	--------------------	-----------

Scoring Model Architecture	Flat six-criteria weighted sum (subject, availability, experience, language/style, feedback, load)	Hierarchical four-bucket composite: $M(s,t) = \alpha * A + \beta * P + \gamma * S + \delta * F$, with sub-weights inside each bucket	The four-bucket model separates concerns, allows independent sub-scorer testing, and maps to Algorithm.md
Subject Eligibility	Weighted term (0.30 weight) in flat sum; non-matching tutors could still rank via other terms	Hard eligibility filter; subject match checked before scoring; non-matching tutors excluded entirely	A tutor who cannot teach the subject should never appear as a candidate
Level Compatibility	Not explicitly defined	$Lvl(s,t) = \max(0, 1 - Ls - Lt / L_{max})$ with $L_{max} = 12$; clamping prevents negative scores	Added to capture grade-level appropriateness; clamping prevents score corruption
Experience Scoring	Mentioned as criterion but no formula	with $\theta = 0.4$ and cold start $Q_t = 0.5$	Years cap prevents unbounded scores; cold start prevents burying new tutors
Style Matching	Mentioned but no scoring method	Cosine similarity on one-hot encoded vectors; zero-norm vectors default to 0.5	One-hot encoding is standard for categorical data; zero-norm fallback handles unspecified preferences
Budget Matching	Pricing mentioned but no formula	$Budget(s,t) = 1$ if rate $\leq$ budget; $\max(0, 1 - (rate - budget)/budget)$ if over;	Zero-division guard prevents crashes when

		zero-division guard	student sets no budget
Region Matching	Not in formal model (only in research notes)	Added as optional term for in-person mode; returns 1 for same region, 0.5 neutral when missing	Missing from seminar's formal model; added for physical tutoring scenarios
Schedule Scoring	Simple overlap ratio as weighted term	with contiguous coverage; zero-availability students rejected as incomplete	Contiguous coverage ensures one tutor slot covers each request; incomplete profiles rejected explicitly
Fairness / Load Balancing	Flat load factor weight (0.05) in sum	with power 1.15 scaling and a +0.05 cold-start boost	Non-linear scaling penalizes high utilization more; boost nudges unused tutors into circulation
Assignment Algorithm	Simple greedy: sort once by score, assign in order	Priority-Queue Greedy with Lazy Fairness Recompute using MaxHeap	Static sort uses fairness data that becomes stale mid-run; lazy recompute keeps correctness at same complexity
Weight Adaptation	Not specified	Proportional renormalization: target weight bumped by η , others proportionally rescaled to keep sum = 1	Seminar's original approach broke normalization; proportional rescaling guarantees sum $w = 1$ always
Feedback Update	Described as updating 'tutor reliability' but no		EMA weights recent feedback more while retaining history; $\lambda =$

	mechanism	with $\lambda = 0.3$ and $FB = \text{rating}/5$	0.3 balances responsiveness to recent sessions with stability of the accumulated rating
Top-K Ranking	Not addressed	Top-K ranking layer for student-facing display (full sort then truncate, per request), plus per-student top-K truncation inside the batch engine	Bounds the response to the top 5–10 tutors and caps batch heap memory at (Section 4.5.3)
Deterministic Tiebreaking	Not addressed	Multi-tier: lower assignedCount then lower tutor ID then FNV-1a hash	Makes output reproducible across runs for testing and debugging
Edge Case Handling	Not addressed	12 explicitly handled edge cases with documented strategies and unit tests	Seminar proposed a theoretical system; implementation required concrete handling of real-world edge cases
Core Architecture	Standard NestJS structure	Framework-independent domain layer (src/core/) isolated from NestJS with dependency inversion	Core algorithm independently testable without NestJS; reusable in other contexts
Evaluation and Benchmarks	Not addressed	Standalone evaluation harness with synthetic datasets, Jain's fairness index, and wall-clock	Provides empirical evidence for theoretical complexity claims

		benchmarking	
Optimality Guarantee	Not addressed	Greedy matching step is provably a 1/2-approximation (exchange argument for weighted bipartite matching), rescoped to the static untruncated step; the deployed engine (fairness in the priority, top-K truncation, tie-breaks, bounded repair) is measured instead, at 97.6-100% of the exact optimum, and an exact min-cost max-flow solver is now built as the evaluation baseline	Defensible lower bound on match quality; min-cost max-flow noted as future work

These 18 changes chart the path from a theoretical seminar proposal to a production-ready implementation. Each change was motivated by one of three things: a correctness issue discovered during implementation (for example, unbounded formulas or missing edge cases), a performance consideration (such as heap-based ranking), or an architectural decision (such as core domain isolation). Throughout the process the Algorithm.md specification was the authoritative reference, and its corrections log gives full traceability for every modification.

Appendix C: Provenance of the Test Suite and Evaluation Figures

The test and evaluation figures in Sections 4.9 and 4.10 are recorded here for transparency. A Jest suite was written for the core matchmaking algorithm in backend/src/core/ together with an evaluation harness, and both were run against the repository, so every figure reflects tests that were written and passed rather than invented numbers.

The suite covers the core classes (AcademicScorer, PreferenceScorer, ScheduleScorer, FairnessScorer, CompositeScorer, GreedyAssignmentEngine, EligibilityFilter, FeedbackUpdater, MaxHeap, VectorMath, weight adaptation and the assignment lifecycle, plus the evaluation harness, the statistics and CSV-contract checks and the stage-3 solver) with unit tests that assert exact expected outputs on fixed inputs derived from the formulas in the code, for example that the composite score equals $\alpha*A + \beta*P + \gamma*S + \delta*F$ for a specific pair.

One dedicated test covers each edge case: zero eligible tutors after filtering, a tutor at zero or full capacity, preference weights not summing to 1, a student with zero availability slots, tied match scores, oversubscription, a new tutor with a null rating, a tutor filled mid-batch, an incremental request at capacity, assignment cancellation, and the structural case in which a student is left out while a seat is held by a student with an acceptable alternative. Each asserts the specific documented behavior, not merely that no exception is thrown.

The suite was run with `pnpm test:cov`, and the terminal output was transcribed into Table 4.9 and Table 4.10. The authoritative run of 28 September 2026 executed 355 tests across 21 suites: 334 passed, 21 were skipped (a single database-dependent integration suite) and none failed.

The harness in `core/evaluation/` ran the deployed engine at the (50,5), (200,20), (1000,100) and (5000,500) student/tutor scales, recording average compatibility score, the percentage of students left unassigned, Jain's fairness index and wall-clock time, with the fairness weight at its designed value and at $\delta = 0$. A second suite (`baseline-statistics.ts`) re-runs the scenario families over 30 independent populations (exploratory seeds 0-29) and reports means with 95% confidence intervals, paired sign tests against the engine, and the exact min-cost max-flow oracle on the same populations.

Tables 4.11 to 4.14 quote those 30-population means rather than the single-population sweep.

The suite and harness were then run against the repository, and the final authoritative run of 28 September 2026 produced `pnpm test` (355 tests run across 21 suites: 334 passed, 21 skipped, including all 197 core-algorithm tests in nine spec files), the harness scenarios, the 30-population baseline-statistics suite including the three-strategy comparison and their output transcribed into Tables 4.9 through 4.19, the three-strategy comparison (Tables 4.11, 4.12 and 4.14), the exact-

oracle comparison (Section 4.5.5) and the optimal-baseline comparison. Because the harness contains no un-seeded randomness, the match-quality and fairness figures in Tables 4.11, 4.12 and 4.14 are exactly reproducible from the repository, while the wall-clock timings in Table 4.13 vary with hardware and are excluded from that claim.

The raw rows behind the figures quoted in Chapter Four are reproduced below, verbatim from the benchmark CSVs checked into the repository at backend/docs/benchmarks/, so that every number in the evaluation can be traced to the run that produced it. Table C.1 is the full scenario sweep of the deployed engine with and without the fairness weight, Table C.2 records the per-run records kept behind the timing means, and Table C.3 is the early capture run of 3 September 2026 that compared the engine against the FCFS-filter, FCFS-best and deferred-acceptance arms on small fixtures before the 30-population suites were run.

Table C.1: Full evaluation sweep, reproduced verbatim (evaluation-results.csv)

scenario	students	tutors	loadFactorWeight	runs	seeds	averageScore	unassignedPercent	jainFairnessIndex	giniLoad	elapsedMeanMs	pairsScored
realistic-seed	50	50	0.05	5	1	0.600392	2.00	0.578554	0.463265	6.6	178
realistic-seed	50	50	0	5	1	0.587922	2.00	0.539551	0.501633	2.9	178
moderate-1.5to1	150	100	0.05	5	1	0.605872	3.33	0.642966	0.406276	23.3	1176
moderate-1.5to1	150	100	0	5	1	0.599263	3.33	0.609420	0.442828	26.6	1183
moderate-2to1	150	75	0.05	5	1	0.600064	8.00	0.697582	0.366377	9.9	914
moderate-2to1	150	75	0	5	1	0.598411	8.00	0.701436	0.363285	10.3	920
moderate-3to1	150	50	0.05	5	1	0.573900	14.67	0.880860	0.198125	8.2	798
moderate-3to1	150	50	0	5	1	0.577783	14.67	0.880860	0.198125	10.3	815
moderate-4to1	200	50	0.05	5	1	0.585125	33.00	0.871650	0.206866	12.9	1087
moderate-4to1	200	50	0	5	1	0.585751	33.00	0.871650	0.206866	11.5	1087

stress-sweep	50	5	0.05	5	1	0.526876	78.00	0.780645	0.290909	0.6	14
stress-sweep	50	5	0	5	1	0.517608	78.00	0.780645	0.290909	0.8	14
stress-sweep	200	20	0.05	5	1	0.596718	79.00	0.747458	0.326190	5.1	396
stress-sweep	200	20	0	5	1	0.591560	79.00	0.747458	0.326190	5.4	396
stress-sweep	1000	100	0.05	5	1	0.712391	75.00	0.833333	0.250000	69.3	6769
stress-sweep	1000	100	0	5	1	0.714494	75.00	0.833333	0.250000	65.1	6769
stress-sweep	5000	500	0.05	5	1	0.780611	75.00	0.833333	0.250000	1855.9	176790
stress-sweep	5000	500	0	5	1	0.785617	75.00	0.833333	0.250000	1879.9	176790

Table C.2: Per-run records for the timing rows (evaluation-per-run-results.csv)

scenario	students	tutors	loadFactor Weight	run	winner	average Score	unassigned Percent	jainFairnessIndex	elapsedMeanMs	pairsScored
moderate-1.5to1	150	100	0.05	1	greedy-engine	0.619380	7.33	0.633475	25.0	1062
moderate-1.5to1	150	100	0.05	2	greedy-engine	0.619380	7.33	0.633475	13.0	1062
moderate-1.5to1	150	100	0	1	da-stable	0.610840	7.33	0.605674	10.0	1062
moderate-1.5to1	150	100	0	2	da-stable	0.610840	7.33	0.605674	9.0	1062

Table C.3: Engine against baseline arms, early capture run of 3 September 2026 (evaluation-capture-results.csv)

scenario	students	tutors	loadFactor Weight	run	winner	fcfs-filter.averageScore	fcfs-best.averageScore	da-stable.averageScore	greedy-engine.averageScore	greedy-engine.unassignedPercent	greedyMs
moderate-1.5to1	12	6	0.05	1	fcfs-best	0.522484	0.566080	0.566080	0.566080	75.00	0
moderate-1.5to1	12	6	0.05	2	fcfs-best	0.522484	0.566080	0.566080	0.566080	75.00	0
moderate-1.5to1	12	6	0	1	fcfs-best	0.503890	0.549781	0.549781	0.549781	75.00	1

moderate-1.5to 1	12	6	0	2	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	0
moderate-2to1	12	6	0.05	1	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	0
moderate-2to1	12	6	0.05	2	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	1
moderate-2to1	12	6	0	1	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	0
moderate-2to1	12	6	0	2	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	1
moderate-3to1	12	6	0.05	1	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	0
moderate-3to1	12	6	0.05	2	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	0
moderate-3to1	12	6	0	1	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	0
moderate-3to1	12	6	0	2	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	1
moderate-4to1	12	6	0.05	1	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	1
moderate-4to1	12	6	0.05	2	fcfs- best	0.522484	0.566080	0.566080	0.566080	75.00	1
moderate-4to1	12	6	0	1	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	0
moderate-4to1	12	6	0	2	fcfs- best	0.503890	0.549781	0.549781	0.549781	75.00	0